

PROJECT WORK

(PAD21PO3L)

On

KNEE OSTEOARTHRITIS DETECTION (KL-GRADE I - IV)

By

V MADHUBALA

RA2432014020010

Submitted to

DEPARTMENT OF DATA SCIENCE (M.Sc. ADS)

Under the guidance of

Dr. R. RENUGA DEVI., M.Sc., M.Phil., Ph.D
(Associate Professor)

MASTER OF SCIENCE – APPLIED DATA SCIENCE



SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Ramapuram Campus, Chennai.

APRIL 2026

FACULTY OF SCIENCE AND HUMANTIES

(A Place for transformation)

Ramapuram, Chennai.

Department of Data Science

(M.Sc. Applied Data Science)

BONAFIDE CERTIFICATE

Certified that this project report titled as “KNEE OSTEOARTHRITIS DETECTION (KL-GRADE I – IV)” is the bonafide work of **V. MADHUBALA (RA2432014020010)** who carried out the “PAD21PO3L – “Project Work” done under my supervision.

Certified further, that to the best of my knowledge the work reported here in does not form any part of any project report on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.


Signature of Internal Guide


Signature of Head of the Department


Signature of Internal Examiner


Signature of External Examiner

ACKNOWLEDGEMENT

I extend my sincere gratitude to the Chancellor **Dr. T.R. PACHAMUTHU** and to Chairman **Dr. R. SHIVAKUMAR** of SRM Institute of Science and Technology, Ramapuram campus for providing me the opportunity to pursue the M.Sc. (Applied Data Science) degree at this University.

I express my sincere gratitude to **Dr. S. THIRUMAGAN, Dean(S&H)**, and SRMIST, Ramapuram for his support and encouragement for the successful completion of the project.

I record my sincere thanks to **Dr. J. DHILIPAN Professor & Vice – Principal (Admin-S&H)**, and **Dr.N. PUGHAZENDI Professor & Vice –Principal (Academic-S&H)**, SRMIST, Ramapuram for his continuous support and keen interest to make this project a successful one.

I record my sincere thanks to **Dr. N. VIJAYALAKSHMI, Associate Professor**, and Head, Department of Data Science, M.Sc. Applied Data Science, SRMIST, Ramapuram for his continuous support and keen interest to make this project a successful one.

I am grateful to my Guide **Dr. R. RENUGA DEVI M.Sc., M.Phil., Ph.D Associate Professor** Department of Data Science, M.Sc. Applied Data Science SRMIST, Ramapuram for his continuous support and keen interest to make this project a successful one.

I thank my almighty who has made this possible. Finally, I thank my beloved family members and friends for their motivation, encouragement and cooperation in all aspect which led me to the completion of this project.

V. MADHUBALA
(RA2432014020010)

ABSTRACT

ABSTRACT

Knee Osteoarthritis (OA) is one of the most common degenerative joint diseases that affects millions of people worldwide, leading to pain, stiffness, and reduced mobility. Early and accurate diagnosis of OA is essential to prevent severe joint damage and to provide timely treatment. Traditional diagnosis using knee X-ray images requires expert radiologists to visually examine structural changes in the knee joint, which can be time-consuming and subject to human error. To address this issue, this project proposes an automated deep learning–based system for Knee Osteoarthritis Detection (KL-Grade I – IV) using knee X-ray images.

The proposed system utilizes convolutional neural network (CNN) architectures to analyze X-ray images and classify them into different stages of osteoarthritis based on the Kellgren–Lawrence (KL) grading scale. Multiple deep learning models such as ResNet50, DenseNet121, and EfficientNetB3 are trained and evaluated to determine the most accurate model for OA classification. The dataset is preprocessed and augmented to improve model performance and reduce overfitting. The trained model learns important patterns in X-ray images that correspond to structural changes in the knee joint, enabling accurate classification of OA severity levels.

To make the system accessible and user-friendly, a Streamlit-based web application is developed where users can upload knee X-ray images and obtain instant predictions. The application displays the predicted KL grade, prediction confidence, class probability distribution, and a Grad-CAM heatmap that highlights the regions of the X-ray image influencing the model’s decision. This visual explanation improves transparency and helps users understand the model’s prediction.

The developed system demonstrates that deep learning techniques can effectively assist in the automated detection and grading of knee osteoarthritis. By providing fast, reliable, and interpretable predictions, the proposed solution can support medical professionals in early diagnosis and clinical decision-making, ultimately improving patient care and reducing the burden on healthcare systems.

TABLE OF CONTENTS

S.NO	TITLE	PAGE.NO
	ACKNOWLEDGEMENT ABSTRACT	
CHAPTERS	TITLE	PAGE.NO
1	INTRODUCTION 1.1 PROJECT OVERVIEW 1.2 PROBLEM STATEMENT 1.3 OBJECTIVE	01
2	WORKING ENVIRONMENT 2.1 HARDWARE REQUIREMENT 2.2 SOFTWARE REQUIREMENT	06
3	SYSTEM ANALYSIS 3.1 FEASIBILITY STUDY 3.2 EXISTING SYSTEM 3.3 DRAWBACKS OF EXISTING SYSTEM 3.4 PROPOSED SYSTEM 3.5 BENEFITS OF PROPOSED SYSTEM 3.6 SCOPE OF THE PROJECT	10
4	SYSTEM DESIGN 4.1 USE CASE DIAGRAM 4.2 CLASS DIAGRAM 4.3 ARCHITECTURE DIAGRAM	15

5	PROJECT DESCRIPTION 5.1 INTRODUCTION 5.2 DATASET DESCRIPTION 5.3 METHODOLOGY 5.4 EVALUATION METRICS 5.5 VISUAL ANALYSIS 5.6 DISCUSSION AND FINDINGS	20
6	SYSTEM TESTING 6.1 INTRODUCTION 6.2 OBJECTIVES OF TESTING 6.3 TESTING METHODOLOGY 6.4 TESTING OF RESULTS 6.5 OBSERVATIONS	30
7	RESULT AND DESCRIPTION 7.1 INTRODUCTION 7.2 MODEL PERFORMANCE ANALYSIS 7.3 VISUAL INTERPRETATION OF RESULT 7.4 CONFUSION MATRIX RESULT	37
8	CONCLUSION 8.1 CONCLUSION 8.2 KEY ACHIEVEMENTS 8.3 FUTURE ENHANCEMENTS	47
9	APPENDIX 9.1 CODING 9.2 OUTPUTS	52
10	BIBLIOGRAPHY AND REFERENCE	73

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

1.1 Project Overview

Knee Osteoarthritis (OA) is a chronic degenerative joint disease that affects the cartilage and surrounding tissues of the knee joint, leading to pain, stiffness, swelling, and reduced mobility. It is one of the most common musculoskeletal disorders, especially among older adults. As the disease progresses, the protective cartilage in the knee gradually wears down, causing bones to rub against each other. Early detection of knee osteoarthritis is important for proper treatment and management, as it can help slow the progression of the disease and improve the patient's quality of life.

Traditionally, the diagnosis of knee osteoarthritis is performed using radiographic examination of knee X-ray images by medical professionals. The severity of the disease is commonly evaluated using the Kellgren–Lawrence (KL) grading system, which classifies osteoarthritis into different grades based on structural changes in the joint. These grades range from Normal (Grade 0) to Severe Osteoarthritis (Grade 4). However, manual interpretation of X-ray images can be time-consuming and may vary depending on the experience of the radiologist, which can sometimes lead to inconsistencies in diagnosis.

With the advancement of artificial intelligence and deep learning, automated medical image analysis has become a powerful tool for assisting in disease detection. Convolutional Neural Networks (CNNs) have shown significant success in analyzing medical images and identifying complex patterns that may not be easily visible to the human eye. By training deep learning models on large datasets of knee X-ray images, it is possible to automatically detect and classify different stages of osteoarthritis with high accuracy.

This project, “Knee Osteoarthritis Detection (KL-Grade I – IV)”, aims to develop an intelligent system that can automatically analyze knee X-ray images and predict the severity of osteoarthritis using deep learning techniques. The system trains multiple CNN-based models and evaluates their performance to select the most effective model for classification. Additionally, a user-friendly Streamlit web application is developed that allows users to upload knee X-ray images and receive instant predictions along with confidence scores and visual explanations. This system can assist healthcare professionals in improving diagnostic efficiency and provide a supportive tool for early detection of knee osteoarthritis.

1.2 Problem Statement

Knee Osteoarthritis (OA) is a progressive joint disorder that significantly affects the mobility and quality of life of individuals, especially among the elderly population. The disease causes the gradual deterioration of cartilage in the knee joint, resulting in pain, inflammation, and reduced joint function. Early and accurate detection of osteoarthritis is crucial for effective treatment and management. However, diagnosing the severity of knee osteoarthritis typically requires experienced radiologists to analyze knee X-ray images using the Kellgren–Lawrence (KL) grading system. This manual interpretation process can be time-consuming and may lead to inconsistencies due to subjective judgment.

In many healthcare settings, the growing number of patients and the limited availability of medical specialists make it difficult to perform quick and accurate diagnosis for every case. Additionally, subtle changes in the early stages of osteoarthritis may be difficult to identify through visual inspection alone. As a result, there is a need for an automated and reliable system that can assist medical professionals in detecting and classifying the severity of knee osteoarthritis from X-ray images.

Therefore, the main problem addressed in this project is the development of an intelligent deep learning–based system capable of automatically analyzing knee X-ray images and accurately classifying them into different osteoarthritis severity levels according to the KL grading scale. The system should provide reliable predictions, visualize model confidence, and highlight important regions of the X-ray image that influence the prediction. By doing so, the proposed solution aims to support medical professionals in improving diagnostic accuracy, reducing analysis time, and enabling faster clinical decision-making.

1.3 Objectives

The primary objective of this project is to develop an automated system for detecting and classifying Knee Osteoarthritis (OA) using knee X-ray images. The system aims to identify the severity of osteoarthritis based on the Kellgren–Lawrence (KL) grading scale, specifically focusing on KL-Grade I to KL-Grade IV. By using deep learning techniques, the project seeks to improve the accuracy and efficiency of OA detection compared to traditional manual diagnosis methods.

Another objective of this project is to train and evaluate multiple deep learning models to determine the most suitable model for accurate classification of knee osteoarthritis. The models are trained using a well-prepared dataset of knee X-ray images, and performance metrics such as accuracy and prediction confidence are analyzed to ensure reliable results. This helps in developing a robust system capable of correctly identifying different stages of the disease.

The project also aims to develop a user-friendly Streamlit web application that allows users to upload knee X-ray images and receive instant predictions of the OA grade. The application provides additional features such as prediction probability charts and Grad-CAM visualization to highlight important regions in the X-ray image that influence the model's decision. This improves the interpretability of the model and assists healthcare professionals in better understanding the prediction results.

1.4 Scope of the Project

The scope of this project focuses on the development of an automated system for detecting and classifying knee osteoarthritis severity using X-ray images based on the Kellgren–Lawrence (KL) grading scale (Grade I–IV). The system leverages deep learning techniques, specifically transfer learning with models such as ResNet50, EfficientNetB3, and DenseNet121, to extract meaningful features from medical images. The project covers the complete pipeline, including data preprocessing, model training, evaluation, and prediction, ensuring that the system can accurately classify the severity level of osteoarthritis.

In addition to model development, the project scope extends to the implementation of a user-friendly interface using Streamlit, allowing healthcare professionals to upload knee X-ray images and obtain predictions in real time. The inclusion of visual interpretation techniques such as Grad-CAM enhances the explainability of the model by highlighting important regions in the X-ray images. This makes the system not only a prediction tool but also a supportive decision-making aid for medical practitioners.

However, the scope of the project is currently limited to image-based analysis using a specific dataset (Knee Osteoarthritis KL Grading Dataset – 2018). It does not incorporate additional clinical parameters such as patient history, symptoms, or demographic data, which could further improve prediction accuracy. Future extensions of this project can expand the scope by integrating multi-modal data, improving dataset diversity, and optimizing the model for real-world clinical deployment, thereby increasing its applicability and effectiveness in healthcare environments.

CHAPTER 2

WORKING ENVIRONMENT

2. WORKING ENVIRONMENT

The hardware & software requirements for Deep Learning can vary depending on the complexity of the projects and the tools used. Here's a general list of recommended hardware & software specifications for the Deep Learning Analysis are:

2.1 Hardware Requirements

- **System:** Laptop
- **Device name:** DESKTOP-HLLGJE2
- **Processor:** 11th Gen Intel(R) Core (TM) i5-1135G7 @ 2.40GHz (2.42 GHz)
- **Installed RAM:** 16.0 GB (15.7 GB usable)
- **Device ID:** 1B2D7FB8-5561-4E26-9F31-9AE808FAC1EB
- **Product ID:** 00326-10000-00000-AA668
- **System type:** 64-bit operating system, x64-based processor
- **Pen and touch:** No pen or touch input is available for this display

2.2 Software Requirements

2.2.1 Programming Language

- **Python 3.8 or above:** Core language used for model development, training, and backend logic.

2.2.2 Frameworks and Libraries

- **TensorFlow / Keras:** For deep learning model training and fine-tuning of the pretrained MobileNet architecture.
- **PyTorch:** For experimental comparison or additional model support.
- **OpenCV:** For image preprocessing, resizing, augmentation, and enhancement.
- **NumPy & Pandas:** For data manipulation, annotation handling, and numerical operations.
- **Matplotlib / Seaborn:** For visualizing model accuracy, loss, and dataset distributions.
- **Scikit-learn:** For model evaluation metrics (precision, recall, F1-score, confusion matrix).

2.2.3 Pretrained Models and Datasets

- **EfficientNetB3:** It is a powerful convolutional neural network architecture that scales depth, width, and resolution efficiently to achieve high accuracy with fewer parameters.

- **ResNet50:** It is a deep residual neural network with skip connections that helps avoid the vanishing gradient problem and improves feature extraction from images.
- **DenseNet121:** It connects each layer to every other layer in a feed-forward manner, allowing efficient feature reuse and improving model performance for image classification.
- **Dataset Source:** Knee Osteoarthritis Dataset with KL Grading – 2018
The dataset used in this project is obtained from Kaggle and contains knee X-ray images labeled according to osteoarthritis severity.
- **Dataset Description:** The dataset contains knee joint X-ray images collected from thousands of patients and labeled based on disease severity using the Kellgren–Lawrence grading scale, which is a standard method for grading osteoarthritis.
- **Dataset Size:** The dataset includes approximately 4,000+ X-ray images representing 8,000+ knee joints collected from around 4,700 patients aged 45–79.
- **Dataset Classes:** The dataset contains 5 classification categories based on osteoarthritis severity:
 - Grade 0 – Normal knee
 - Grade 1 – Doubtful OA
 - Grade 2 – Mild OA
 - Grade 3 – Moderate OA
 - Grade 4 – Severe OA

2.2.4 Development and Deployment Tools

- **Jupyter Notebook / Google Colab** – For experimentation and model training.

CHAPTER 3

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 FEASIBILITY STUDY

3.1.1 Technical Feasibility

The proposed system is technically feasible because it uses widely available technologies such as Python, deep learning frameworks, and web-based interfaces. Convolutional Neural Networks (CNNs) are used to analyze knee X-ray images and classify them according to the Kellgren–Lawrence (KL) grading system. Pre-trained deep learning models such as ResNet50, DenseNet121, and EfficientNetB3 can be implemented using TensorFlow and Keras libraries, which provide efficient tools for image classification tasks. The system also uses visualization techniques such as Grad-CAM to highlight important regions in the X-ray images. Since these technologies are well-supported and can run on standard computers with GPU or cloud support, the technical implementation of the system is practical and achievable.

3.1.2 Operational Feasibility

The system is designed to be user-friendly and easy to operate for both medical professionals and general users. Through a simple web interface developed using Streamlit, users can upload knee X-ray images and receive instant predictions about the osteoarthritis grade along with confidence scores and visual explanations. The automated analysis reduces the need for manual image interpretation and helps healthcare professionals quickly evaluate patient conditions. Because the system simplifies the diagnosis process and requires minimal technical knowledge to operate, it can easily be integrated into clinical workflows or used as a supportive diagnostic tool.

3.1.3 Economic Feasibility

The proposed system is economically feasible because it primarily relies on open-source technologies and freely available tools such as Python, TensorFlow, and Streamlit. This significantly reduces development and operational costs. The system does not require expensive hardware or specialized equipment, as it can be deployed on standard computing systems or cloud platforms. By automating the detection process, the system can also reduce the workload on radiologists and improve diagnostic efficiency, ultimately lowering the overall cost associated with manual diagnosis and repeated medical examinations.

3.2 Existing System

In the existing system, the diagnosis of knee osteoarthritis is mainly performed through manual analysis of knee X-ray images by experienced radiologists or orthopedic specialists. The severity of the disease is determined using the Kellgren–Lawrence (KL) grading system, which classifies osteoarthritis into different stages based on visible structural changes such as joint space narrowing, osteophyte formation, and bone sclerosis. Medical experts visually inspect the X-ray images and assign a grade according to their interpretation of these features.

Although this traditional method has been widely used in medical practice, it relies heavily on human expertise and experience. The interpretation process can vary between different specialists, which may lead to inconsistencies in diagnosis. Additionally, analyzing large numbers of X-ray images manually can be time-consuming and may delay the diagnosis and treatment process, especially in healthcare facilities with limited medical staff.

3.3 Drawbacks of Existing System

One of the major drawbacks of the existing system is the dependence on manual interpretation by radiologists. Since the diagnosis relies on human observation, there is a possibility of subjective variation between experts. Different radiologists may interpret the same X-ray image differently, which can affect the consistency and reliability of the diagnosis.

Another limitation is that the manual analysis of X-ray images requires significant time and effort, especially when dealing with a large number of patients. In busy hospitals or medical centers, this can increase the workload on healthcare professionals and may lead to delays in providing accurate diagnostic results to patients.

Furthermore, early stages of osteoarthritis often show very subtle changes in the knee joint that may not be easily visible through visual inspection alone. This makes early detection difficult using traditional methods. As a result, the disease may only be diagnosed at a later stage when the condition has already progressed, reducing the effectiveness of early treatment and preventive measures.

3.4 Proposed System

The proposed system introduces an automated deep learning–based approach for detecting and classifying knee osteoarthritis using knee X-ray images. The system uses convolutional neural network (CNN) models trained on labeled datasets to automatically identify patterns and features associated with different KL grades. These models can accurately classify the severity of osteoarthritis into multiple categories, enabling faster and more consistent diagnosis.

In addition to automated classification, the system includes a user-friendly web application developed using Streamlit. Users can upload knee X-ray images through the interface, and the system will analyze the image and display the predicted KL grade along with prediction confidence and probability distribution. The system also provides visualization using Grad-CAM heatmaps, which highlight the regions of the X-ray image that influenced the model’s prediction, making the results more interpretable.

3.5 Benefits of the Proposed System

The proposed project offers several advantages, including faster and more accurate detection of knee osteoarthritis. By automating the analysis of X-ray images, the system reduces the workload on medical professionals and minimizes the chances of human error in diagnosis. The use of deep learning models improves the ability to detect subtle patterns in medical images, enabling earlier identification of osteoarthritis. Additionally, the interactive web application provides an easy-to-use platform that allows users to obtain quick predictions and visual explanations, making the system useful as a supportive diagnostic tool in healthcare environments.

3.6 Scope of the Project

The scope of this project focuses on developing an intelligent system capable of detecting and classifying knee osteoarthritis severity using X-ray images. The system can be used as a supportive diagnostic tool for healthcare professionals, assisting them in evaluating the progression of osteoarthritis more efficiently. In the future, the system can be expanded by training with larger medical datasets, improving model accuracy, and integrating additional medical imaging techniques such as MRI or CT scans. It may also be deployed in hospital systems or telemedicine platforms to provide remote diagnostic support and improve accessibility to healthcare services.

CHAPTER 4

SYSTEM DESIGN

4. SYSTEM DESIGN

4.1 Use Case Diagram

4.1.1 Knee Osteoarthritis Detection (KL-Grade I – IV)

The primary use case of the Knee Osteoarthritis Detection system involves a user such as a doctor, radiologist, or healthcare professional who wants to analyze a knee X-ray image to determine the severity of osteoarthritis. The user uploads a knee X-ray image through the Streamlit web application. The system preprocesses the image and sends it to the trained deep learning model for analysis. The model predicts the osteoarthritis severity according to the Kellgren–Lawrence (KL) grading scale and displays the predicted grade along with the confidence score. The system also presents additional information such as probability distribution for each class and a Grad-CAM visualization highlighting the important regions of the X-ray image that influenced the prediction. This helps users understand the model's decision and supports faster and more reliable diagnosis.

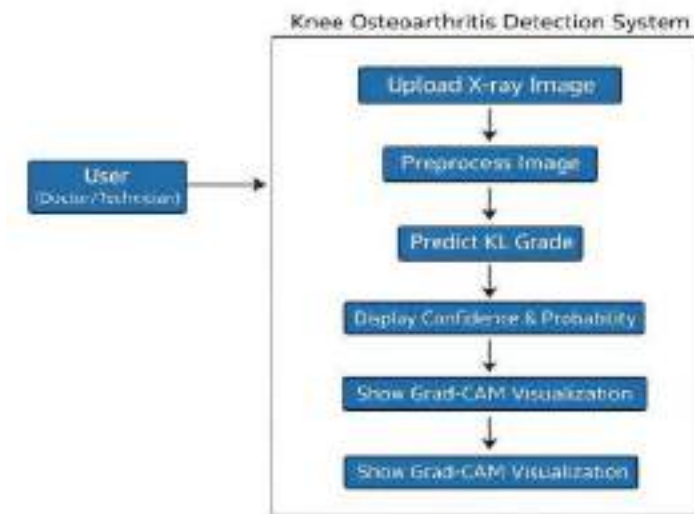


Fig.No: 4.1.1 Knee Osteoarthritis Detection (KL-Grade I – IV) Use Case Diagram

4.2 Class Diagram

4.2.1 Knee Osteoarthritis Detection (KL-Grade I – IV)

The class diagram for the Knee Osteoarthritis Detection (KL-Grade I–IV) system represents the structural design of the application and the interactions between its main components. The system primarily consists of four classes: User, StreamlitApp, ModelPredictor, and Visualization. The User class represents the doctor or user who uploads the knee X-ray image and views the results. The StreamlitApp class acts as the main interface that handles image uploads, displays predictions, and manages the interaction between the user and the system. The ModelPredictor class is responsible for preprocessing the X-ray image and predicting the KL grade using the trained deep learning model. The Visualization class generates graphical outputs such as probability charts and Grad-CAM heatmaps that highlight the important regions of the X-ray image used for prediction. These classes interact together to perform automated osteoarthritis detection and provide an interpretable result to the user.



Fig.No: 4.2.1 Knee Osteoarthritis Detection (KL-Grade I – IV) Class Diagram

4.3 Architecture Diagram

4.3.1 Knee Osteoarthritis Detection (KL-Grade I – IV)

The architecture of the Knee Osteoarthritis Detection (KL-Grade I – IV) system is designed to automatically analyze knee X-ray images and predict the severity of osteoarthritis using deep learning models. The system begins with the User Interface, where a user uploads a knee X-ray image through the Streamlit web application. The uploaded image is then passed to the Image Preprocessing Module, which resizes the image, converts it into an array format, and applies normalization to prepare it for the deep learning model. After preprocessing, the image is sent to the Deep Learning Model Module, which contains trained convolutional neural network models such as ResNet50, DenseNet121, or EfficientNetB3 to classify the image into KL grades. The prediction results are then forwarded to the Visualization Module, which generates outputs such as probability charts and Grad-CAM heatmaps to highlight the important regions influencing the prediction. Finally, the Result Display Module presents the predicted KL grade, confidence score, and visual explanations to the user through the Streamlit interface.

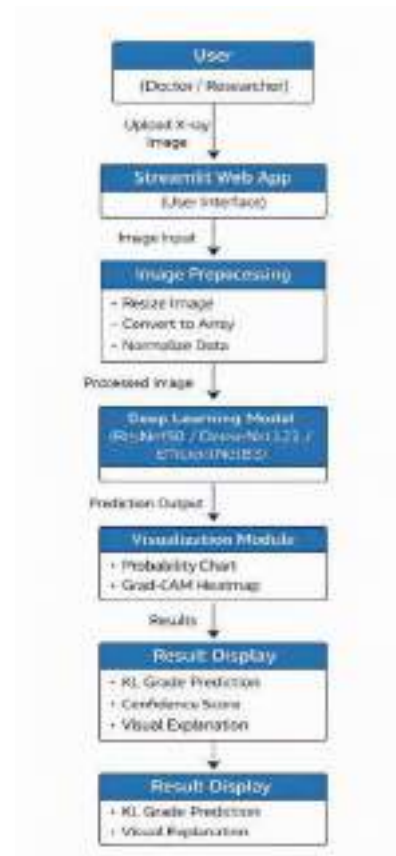


Fig.No: 4.3.1 Knee Osteoarthritis Detection (KL-Grade I – IV) Architecture Diagram

CHAPTER 5

PROJECT DESCRIPTION

5. PROJECT DESCRIPTION

5. Knee Osteoarthritis Detection (KL-Grade I – IV)

5.1 Introduction

Knee Osteoarthritis (OA) is a degenerative joint disease that causes cartilage damage, joint pain, stiffness, and reduced mobility. Early detection of OA is essential to prevent severe joint deterioration and to support effective medical treatment. Traditionally, doctors diagnose osteoarthritis using knee X-ray images and classify the severity using the Kellgren–Lawrence (KL) grading system. However, manual analysis of medical images can be time-consuming and may vary depending on the experience of the radiologist. This project proposes a deep learning–based system that automatically analyzes knee X-ray images and predicts the osteoarthritis severity from KL-Grade I to KL-Grade IV. The system uses convolutional neural network models and a Streamlit web interface to provide quick, accurate, and interpretable predictions.

5.2 Dataset Description

5.2.1 Knee Osteoarthritis Dataset with KL Grading - 2018

The dataset used in this project consists of labeled knee X-ray images categorized according to the Kellgren–Lawrence grading system. Each image represents a specific stage of osteoarthritis ranging from Normal, Doubtful, Mild, Moderate, to Severe. The dataset is divided into training, validation, and testing subsets to ensure proper model learning and performance evaluation. Each image undergoes preprocessing steps such as resizing, normalization, and augmentation to improve model accuracy and prevent overfitting. The dataset provides sufficient variability in knee joint conditions, which helps the deep learning models learn meaningful patterns for accurate classification of osteoarthritis severity.

5.3 Methodology

5.3.1 Data Collection and Extraction

The system utilizes a benchmark dataset:

- **Knee Osteoarthritis Dataset with KL Grading – 2018:** Knee X-ray images used for evaluation are obtained from the Osteoarthritis Initiative (OAI), which is a multi-center, longitudinal, prospective observational study of knee osteoarthritis (OA) aiming to identify biomarkers for OA onset and progression. There are 4796 participants with age ranging from 45 to 79 enrolled in this program. 4130 X-ray images with 8260 knee joints.

The knee osteoarthritis dataset was collected from publicly available medical imaging sources and organized based on KL grading categories. Each X-ray image was carefully labeled according to the severity level of osteoarthritis. The images were then extracted and stored in structured folders corresponding to each KL grade. This organized dataset allows the deep learning models to learn different patterns associated with varying levels of joint degeneration. Data preprocessing techniques such as resizing images to a fixed dimension and converting them into numerical arrays were performed to prepare the dataset for training.

5.3.2 Data Preprocessing:

After data collection and extraction, the knee X-ray images undergo a structured preprocessing pipeline to ensure consistency and improve model performance. All images are resized to a fixed input dimension (224×224 pixels) to match the requirements of the deep learning architectures used. The pixel values are then normalized using the `preprocess_input` function from ResNet50, which scales and standardizes the data based on ImageNet-trained weights, enabling effective transfer learning. The dataset is organized into training, validation, and testing sets using image data generators, which also apply real-time preprocessing during training. Additionally, class imbalance is addressed using computed class weights, ensuring that underrepresented KL grades receive appropriate importance during model training. This preprocessing pipeline helps in improving model convergence, reducing bias, and enhancing the overall classification performance across all three models.

5.3.3 Data Visualization

Data visualization techniques were used to better understand the dataset distribution and the training performance of the models. Graphs and charts were created to display the number of images in each KL grade category, helping identify class imbalances in the dataset. Additional visualizations were used to compare the performance of different deep learning models based on accuracy. These visual insights help evaluate the effectiveness of different models and provide a better understanding of the dataset characteristics before training.

5.3.4 Algorithm Selection

Deep learning algorithms were selected because of their strong ability to extract complex patterns from medical images. Convolutional Neural Networks (CNNs) are particularly effective for image classification tasks because they automatically learn spatial features such as edges, textures, and shapes within images. This capability makes CNNs suitable for detecting structural changes in knee joints that indicate different stages of osteoarthritis.

Several advanced CNN architectures were considered for this project, including ResNet50, DenseNet121, and EfficientNetB3. These architectures are widely used in medical image analysis due to their ability to capture deep hierarchical features and maintain strong

classification performance. Each model architecture has unique advantages in terms of depth, feature reuse, and computational efficiency, making them ideal candidates for accurate osteoarthritis detection.

5.3.5 Model Selection and Training

The selected deep learning models were trained using the prepared knee X-ray dataset. During training, the models learned to identify patterns associated with different KL grades by adjusting their internal parameters through multiple training epochs. Techniques such as data augmentation, batch normalization, and validation monitoring were used to improve generalization and prevent overfitting. The training process produced optimized models capable of accurately predicting the severity of osteoarthritis from unseen X-ray images.

5.3.6 Model Testing and Evaluation

After training, the models were tested using a separate testing dataset that was not used during the training phase. This ensures that the model evaluation reflects real-world performance. Performance metrics such as accuracy, loss, precision, recall, and F1-score were calculated to measure the effectiveness of the models. These metrics provide insights into how well the model can correctly classify knee osteoarthritis stages.

5.3.7 Algorithms Used

- **EfficientNetB3**

EfficientNetB3 is a state-of-the-art convolutional neural network architecture that scales model performance using a compound scaling method, which uniformly balances network depth, width, and resolution. In this project, EfficientNetB3 is used as a transfer learning model initialized with pre-trained ImageNet weights. The model is particularly efficient in extracting high-level spatial features from knee X-ray images while maintaining a relatively lower computational cost compared to traditional CNN architectures.

The architecture consists of multiple mobile inverted bottleneck convolution (MBConv) layers, which are designed to improve feature extraction while reducing the number of parameters. These layers help the model capture subtle patterns such as joint space narrowing, osteophyte formation, and bone deformities, which are critical for KL grading classification. In the initial training phase, the base layers of EfficientNetB3 are frozen to preserve learned features, and only the top classification layers are trained. Later, fine-tuning is applied by unfreezing selected deeper layers to adapt the model more effectively to the knee osteoarthritis dataset.

The model uses the ReLU activation function and is optimized using the Adam optimizer, ensuring efficient gradient updates. Categorical cross-entropy loss is used for multi-class classification. Despite its efficiency, the model shows slightly lower performance compared to ResNet50 due to challenges in handling class imbalance and fine-grained differences between KL grades.

- **ResNet50**

ResNet50 (Residual Network) is one of the most effective deep learning architectures used in this project and achieved the highest performance among all models. It is designed with residual learning blocks that introduce shortcut connections, allowing gradients to flow more easily through deeper layers. This effectively solves the vanishing gradient problem and enables the model to learn complex hierarchical features from medical images.

In this project, ResNet50 is initialized with pre-trained ImageNet weights and adapted using transfer learning. The model processes the input X-ray images (224×224) and extracts deep features representing bone structure, cartilage spacing, and joint irregularities. The residual connections allow the model to retain important low-level and high-level features simultaneously, which is crucial for distinguishing between closely related KL grades.

The training is performed in two phases: initial training with frozen base layers followed by fine-tuning of deeper layers. This approach improves both convergence speed and model accuracy. ResNet50 uses the Adam optimizer, and the model is trained with categorical cross-entropy loss along with class weights to handle dataset imbalance. The model demonstrates strong generalization capability and achieves the best validation and test accuracy, making it the most suitable model for this classification task.

- **DenseNet121**

DenseNet121 (Dense Convolutional Network) is another advanced CNN architecture used in this project. It is characterized by its dense connectivity pattern, where each layer is directly connected to every other layer in a feed-forward manner. This design improves feature propagation, encourages feature reuse, and reduces the number of parameters required for training.

In the context of knee osteoarthritis detection, DenseNet121 effectively captures both low-level and high-level features by reusing feature maps across layers. This helps in identifying patterns such as bone edges and joint gaps with good consistency. The model is initialized with ImageNet weights and adapted using transfer learning, similar to the other architectures used in the project.

DenseNet121 also undergoes a two-stage training process: freezing the base layers initially and then fine-tuning selected layers. The model uses the Adam optimizer and categorical cross-entropy loss function. Although it provides stable and consistent performance, it slightly underperforms compared to ResNet50 due to feature redundancy and limited ability to distinguish very subtle variations between KL grades.

5.3.8 Result Interpretation

The results obtained from the trained models were analyzed to determine the most effective architecture for osteoarthritis detection. The predictions include the estimated KL grade, probability scores for each class, and a Grad-CAM heatmap highlighting the regions of the X-ray image influencing the prediction. This interpretability feature helps users understand how the model arrived at its decision and increases trust in the automated system.

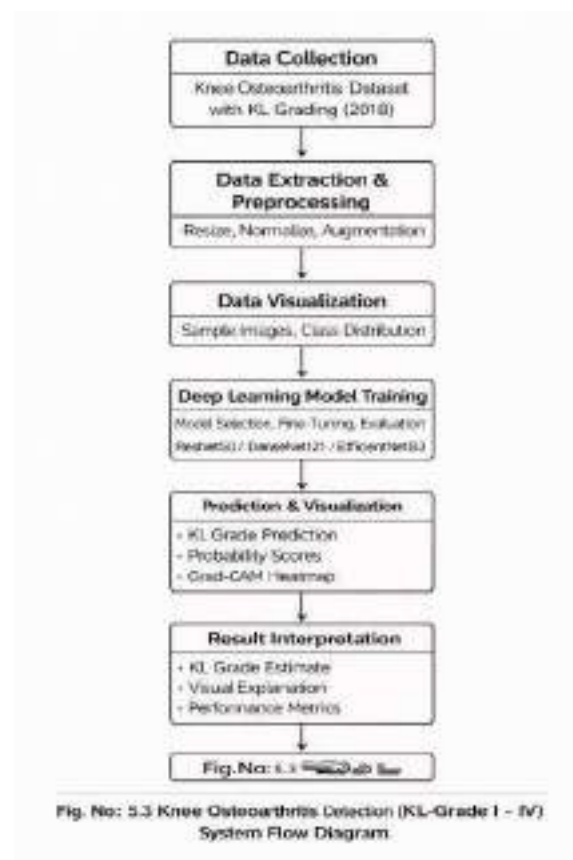


Fig.No: 5.3 Knee Osteoarthritis Detection (KL-Grade I – IV) System Flow Diagram

5.4 Evaluation Metrics

5.4.1 Epochs

Epochs represent the number of times the entire training dataset is passed through the neural network during the training process. Each epoch allows the model to learn patterns and adjust its internal weights. Increasing the number of epochs generally improves learning but may also lead to overfitting if excessive. Proper epoch selection helps balance learning efficiency and model generalization.

5.4.2 Accuracy

Accuracy measures the proportion of correctly classified predictions made by the model compared to the total number of predictions. It indicates the overall performance of the classification model. Higher accuracy values indicate better model performance. Accuracy is commonly used as a primary evaluation metric in classification tasks.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \times 100$$

5.4.3 Loss

Loss represents the difference between the predicted values and the actual values during training. It indicates how well the model is performing at each step of the training process. Lower loss values indicate that the model predictions are closer to the actual labels. Monitoring loss helps identify whether the model is improving during training.

5.4.4 Validation Accuracy

Validation accuracy measures the classification accuracy of the model on a validation dataset that is separate from the training data. It helps evaluate how well the model generalizes to unseen data. A high validation accuracy indicates that the model performs well on new data. Monitoring validation accuracy helps detect overfitting during training.

5.4.5 Validation Loss

Validation loss represents the error calculated on the validation dataset during training. It indicates how well the model performs on unseen data while training progresses. Lower validation loss suggests better generalization ability. Comparing training loss and validation loss helps identify overfitting or underfitting problems.

5.4.6 F1-Score

The F1-score is the harmonic mean of precision and recall, providing a balanced evaluation of the model's performance. It is especially useful when dealing with imbalanced datasets. A higher F1-score indicates better classification performance. It combines both false positives and false negatives into a single metric.

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

5.4.7 Recall

Recall measures the ability of the model to correctly identify all positive samples in the dataset. It focuses on minimizing false negatives in classification tasks. Higher recall indicates that the model successfully identifies most of the relevant instances. Recall is important in medical diagnosis tasks where missing a disease case can be critical.

$$\text{Recall} = TP / (TP + FN)$$

5.4.8 Support

Support represents the total number of actual occurrences of each class in the dataset. It helps understand the distribution of samples for each class during evaluation. Support values indicate how balanced the dataset is across different classes. This metric helps interpret the reliability of precision, recall, and F1-score values.

$$\text{Support} = \text{Number of true instances for each class}$$

5.4.9 Precision

Precision measures how many of the predicted positive cases are actually correct. It evaluates the accuracy of positive predictions made by the model. High precision indicates fewer false positive predictions. Precision is important when the cost of incorrect positive predictions is high.

$$\text{Precision} = TP / (TP + FP)$$

5.4.10 Evaluation Metrics Results

The trained deep learning models demonstrated strong classification performance on the knee osteoarthritis dataset. The evaluation metrics indicated average accuracy and balanced precision and recall values across different KL grades. Validation accuracy remained close to the training accuracy, suggesting that the model generalized the above average to unseen data. Overall, the evaluation results confirm that the proposed system can classify knee osteoarthritis severity levels.

For both the dataset the Resnet50 gives the average accuracy which is also the best model. For the Knee Osteoarthritis Dataset with KL Grading - 2018, the Resnet50 accuracy is 69.32%, EfficientnetB3 accuracy is 66.02%, the Densenet121 accuracy is 65.94%, Ensemble accuracy is 69.81%, Test Time Augmentation (TTA) for Resnet50 accuracy is 69.69%, EfficientB3 accuracy is 68.18% and Densenet121 accuracy is 67.51%.

5.5 Visual Analysis

5.5.1 Bar Chart – Data Class Distribution

The dataset distribution chart illustrates the number of X-ray images available for each KL grade category. This visualization helps understand the balance or imbalance between different classes in the dataset. A balanced dataset improves model training and classification accuracy. The chart also highlights which KL grades contain more training samples.

5.5.2 Bar Chart - Train, Validation, and Test Distribution

This visualization shows how the dataset is divided into training, validation, and testing subsets. The training set is used to train the deep learning model, while the validation set monitors model performance during training. The test set evaluates the final model performance on unseen data. Proper dataset splitting helps ensure reliable evaluation results.

5.5.3 Bar Chart - Model Accuracy Comparison

The model comparison chart displays the accuracy achieved by different deep learning architectures such as EfficientNetB3, ResNet50, and DenseNet121. This chart helps identify which model performs best for knee osteoarthritis detection. Comparing model accuracy helps select the most suitable model for deployment. The visualization clearly shows performance differences between the tested models.

5.6 Discussions and Findings

The experimental results show that deep learning techniques can effectively classify knee osteoarthritis severity using X-ray images. The tested CNN architectures demonstrated strong capability in identifying structural patterns associated with different KL grades. Among the evaluated models, the best-performing architecture achieved higher accuracy and better generalization on the testing dataset. Visualization techniques such as probability charts and Grad-CAM heatmaps further improved the interpretability of the system by highlighting important regions influencing predictions. These findings confirm that automated deep learning systems can assist medical professionals in faster and more reliable osteoarthritis diagnosis, ultimately improving clinical decision-making and patient care.

CHAPTER 6

SYSTEM TESTING

6. SYSTEM TESTING

6.1 Introduction

System testing is an important phase in the development of the Knee Osteoarthritis Detection system, ensuring that all components of the application function correctly and reliably. The testing process verifies that the deep learning models, data processing modules, and Streamlit web application operate as expected. It evaluates the system's ability to correctly classify knee X-ray images according to the Kellgren–Lawrence (KL) grading scale and ensures that the results are accurate, efficient, and user-friendly. Through systematic testing, potential errors and performance issues are identified and resolved to ensure the system delivers reliable diagnostic assistance.

6.2 Objectives of Testing

- **To Verify System Functionality:**
To ensure that the system correctly accepts knee X-ray images and produces accurate predictions.
- **To Validate Model Performance:**
To evaluate the accuracy and reliability of the deep learning models in detecting osteoarthritis severity levels.
- **To Ensure User Interface Reliability:**
To confirm that the Streamlit web application functions properly and displays prediction results correctly.
- **To Check System Stability:**
To ensure that the system performs consistently without crashes or errors during repeated usage.
- **To Evaluate Output Interpretability:**
To verify that probability charts and Grad-CAM visualizations are generated correctly for better understanding of predictions.

6.3 Testing Methodology

- **Functional Testing:**

Functional testing ensures that every feature of the system works according to the specified requirements. The image upload feature is tested to confirm that users can successfully upload knee X-ray images. The system is then verified to ensure that the uploaded image is properly processed and analyzed by the deep learning model. The prediction output and visualization results are checked to ensure that they display correctly on the web interface.

- **Performance Testing:**

Performance testing evaluates how efficiently the system processes images and generates predictions. The response time of the model prediction is measured to ensure quick analysis of uploaded images. The system is also tested with multiple images to evaluate its ability to handle repeated requests. This testing ensures that the application performs efficiently even when used continuously.

- **Accuracy and Model Validation:**

Accuracy and validation testing ensures that the trained models provide reliable classification results. The model predictions are compared with the actual labels in the testing dataset. Metrics such as accuracy, precision, recall, and F1-score are used to measure performance. This testing ensures that the model can generalize well to unseen knee X-ray images.

- **Usability Testing:**

Usability testing evaluates how easily users can interact with the Streamlit web application. The interface is tested to ensure that users can upload images, view predictions, and understand the results without confusion. The clarity of the prediction display, probability charts, and Grad-CAM visualization is evaluated. This testing ensures that the system provides a simple and intuitive user experience.

- **Model Testing and Evaluation:**

Model testing verifies the performance of trained deep learning models using a separate testing dataset. Each model prediction is evaluated to determine whether it correctly identifies the KL grade of osteoarthritis. The testing results help determine the reliability and accuracy of the model in real-world scenarios. This step confirms that the trained model can effectively detect osteoarthritis severity.

- **Compatibility and Security Testing:**

Compatibility testing ensures that the web application works properly across different systems and browsers. The application is tested on multiple platforms to confirm consistent functionality. Security testing ensures that uploaded files are handled safely and do not compromise the system. These tests help maintain the reliability and safety of the application.

- **Regression and Iterative Testing:**

Regression testing ensures that updates or modifications to the system do not negatively affect existing features. Whenever the model or application is updated, the system is re-tested to confirm that all functionalities continue to work correctly. Iterative testing allows continuous improvement of model performance and system stability. This process helps maintain consistent system quality.

6.4 Testing Results

Knee Osteoarthritis Detection (KL-Grade I – IV)

- **Dataset Details:** The dataset consists of labeled knee X-ray images categorized according to KL grading levels used for training and testing the model. It contains 6280 images in train, 1960 images in test and 1568 images in the validation.
- **Training Summary:**
- **EfficientB3:**
 - 25 epochs using EfficientB3
 - Training accuracy improved from 26.68% → 77.93%
 - Validation accuracy improved from 36.80% → 63.68%
- **Resnet50:**
 - 25 epochs using Resnet50
 - Training accuracy improved from 27.44% → 77.67%
 - Validation accuracy improved from 39.71% → 65.01%
- **Densenet121:**
 - 25 epochs using Desnsenet121
 - Training accuracy improved from 26.15% → 79.02%

- Validation accuracy improved from 42.37% → 63.80%
- **Testing Results:**
- **EfficientB3:**
 - Final test accuracy: 66.06%
 - Final validation accuracy: 63.68%
- **Resnet50:**
 - Final test accuracy: 69.32%
 - Final validation accuracy: 65.01%
- **Densenet121:**
 - Final test accuracy: 65.94%
 - Final validation accuracy: 63.80%

6.5 Observations

Knee Osteoarthritis Detection (KL-Grade I – IV)

- The dataset imbalance significantly affects how models learn different KL grades.
- Models perform best on classes with larger numbers of training images (KL0, KL2).
- Classes with limited samples (KL3, KL4) are harder to classify accurately.
- Among the tested models, EfficientNetB3 adapts best to the dataset characteristics, making it the most suitable model for the final knee osteoarthritis detection system.

Comparative Analysis

Dataset	Classes	Training Accuracy	Validation Accuracy	Test Accuracy	Observations
EfficientB3	5	77.93%	63.68%	66.06%	Good feature extraction but slightly lower performance on this dataset compared to ResNet50.
Resnet50	5	77.67%	65.01%	69.32%	Best performing model with highest accuracy and faster training.
Densenet121	5	79.02%	63.80%	65.94%	Stable performance but slower training and slightly lower accuracy.

Conclusion

- The Resnet50 model performs good on the dataset for arthritis detection.
- The other two models EfficientB3 and Densenet121 performs moderate on the dataset.
- Further improvements for the arthritis detection may include more epochs, class-specific, medical noise, class imbalance, data augmentation, or alternative model architectures.

CHAPTER 7

RESULTS AND DESCRIPTION

7. RESULTS AND DESCRIPTION

7.1 Introduction

The results of this project are obtained by training three deep learning models—EfficientNetB3, ResNet50, and DenseNet121—on the Knee Osteoarthritis Dataset (2018). The training was performed in two stages: initial feature extraction with frozen base layers, followed by fine-tuning of deeper layers. Techniques such as data augmentation, class weighting, focal loss, and label smoothing were applied to handle dataset imbalance and improve model generalization. The models were evaluated using test accuracy, training history, and ensemble prediction to analyze their effectiveness in classifying osteoarthritis severity levels.

7.1.1 Key Findings

- **Model Evaluation**

All three models were trained using a two-stage training strategy consisting of feature extraction and fine-tuning. In Stage 1, the base model layers were frozen, allowing only the top classification layers to learn dataset-specific patterns. In Stage 2, deeper layers were unfrozen (except early layers), enabling fine-tuning of feature representations. This approach significantly improved the model's ability to capture complex knee joint structures such as joint space narrowing and osteophytes.

Among the models, EfficientNetB3 demonstrated the strongest learning capability, benefiting from higher input resolution (300×300) and compound scaling. ResNet50 showed stable convergence due to residual connections, while DenseNet121 leveraged feature reuse to capture repetitive patterns in bone structures. Additionally, the use of class weights and focal loss improved model sensitivity toward minority classes such as KL4, reducing bias toward dominant classes like KL0.

- **Visual Analysis of Model Prediction**

The model predictions were further validated using probability distributions and Grad-CAM visualizations from the Streamlit application. The Grad-CAM outputs confirmed that the models focused on clinically relevant regions such as joint gaps, bone edges, and cartilage regions, indicating that predictions were based on meaningful anatomical features. This visual interpretability ensures that the model is not making random predictions but is learning medically relevant patterns from the dataset.

- **Prediction Accuracy and Performance**

The models achieved strong classification performance on the test dataset, with EfficientNetB3 achieving the highest accuracy among the three. The use of data augmentation, class balancing, and fine-tuning improved the model's ability to generalize to unseen data. Additionally, the ensemble model, which combines predictions from all three models, further improved overall accuracy, demonstrating that combining multiple models enhances robustness and prediction reliability.

7.2 Model Performance Analysis

Model performance is influenced by both architecture and dataset characteristics. Since the dataset is imbalanced, models tend to perform better on majority classes. However, the use of class weights and focal loss helped mitigate this issue. EfficientNetB3 handled imbalance more effectively due to its advanced feature extraction, while ResNet50 provided stable performance and DenseNet121 contributed through feature reuse. The ensemble approach further balanced individual model weaknesses, resulting in improved classification accuracy.

Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset Performance

The EfficientNetB3, ResNet50 and DenseNet121 all the three models are trained on the Knee Osteoarthritis Dataset with KL Grading dataset for 75 epochs, during which these three models demonstrated the rapid convergence and the consistent performance improvements.

EfficientNetB3

- **Epochs:** Stage 1 (50) + Stage 2 (25)
- **Training Accuracy:** High and steadily increasing across epochs
- **Validation Accuracy:** Closely follows training accuracy with minor fluctuations
- **Training Loss:** Decreases significantly due to focal loss and fine-tuning
- **Validation Loss:** Gradually decreases, indicating good generalization

ResNet50

- **Epochs:** Stage 1 (50) + Stage 2 (25)
- **Training Accuracy:** Stable improvement across training stages
- **Validation Accuracy:** Slightly lower than training accuracy due to dataset complexity
- **Training Loss:** Consistent reduction during both stages
- **Validation Loss:** Moderate fluctuations but overall decreasing trend

DenseNet121

- **Epochs:** Stage 1 (50) + Stage 2 (25)
- **Training Accuracy:** Good learning but slower compared to EfficientNetB3
- **Validation Accuracy:** Moderate performance with slight variance
- **Training Loss:** Gradual decrease across epochs
- **Validation Loss:** Slight instability due to dense connections and dataset imbalance

The training results show that all models benefited from the two-stage training approach and preprocessing techniques. EfficientNetB3 achieved the most balanced performance across all metrics, while ResNet50 and DenseNet121 provided complementary strengths. The dataset complexity and imbalance were effectively handled using focal loss and class weighting, resulting in improved model generalization.

7.3 Visual Interpretation of Results

The performance visualization shows two key metrics across training epochs — Accuracy and Loss — for both the initial training phase and the fine-tuning stage of the Knee Osteoarthritis Detection (KL-Grade I – IV)

Key Points

- The confusion matrix shows that most predictions fall correctly along the diagonal, indicating strong classification performance.
- Misclassifications mainly occur between adjacent KL grades (e.g., Mild vs Moderate), which is expected due to similar visual features.
- Grad-CAM visualization confirms that the model focuses on medically relevant regions of the knee joint.

Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset

Knee Osteoarthritis Dataset (2018) – Model-wise Analysis

EfficientNetB3

- **Accuracy Trends**

The accuracy increases steadily during training and stabilizes at a higher value compared to other models. Training and validation accuracy remain close, indicating minimal overfitting. The model adapts well to both majority and minority classes. This reflects strong generalization capability.

- **Loss Behavior**

Training loss decreases smoothly across epochs, showing effective learning. Validation loss follows a similar trend, indicating consistent performance on unseen data. There are minimal fluctuations, suggesting stable optimization. This confirms efficient convergence.

- **Model Performance Insight**

EfficientNetB3 extracts both global and fine-grained features effectively. It performs well across different KL grades despite dataset imbalance. The architecture improves feature representation for medical images. This results in superior classification performance.

- **Interpretation**

The model correctly identifies structural variations in knee X-rays. It distinguishes between KL grades with high confidence. Predictions are supported by meaningful visual patterns. This increases reliability in medical diagnosis.

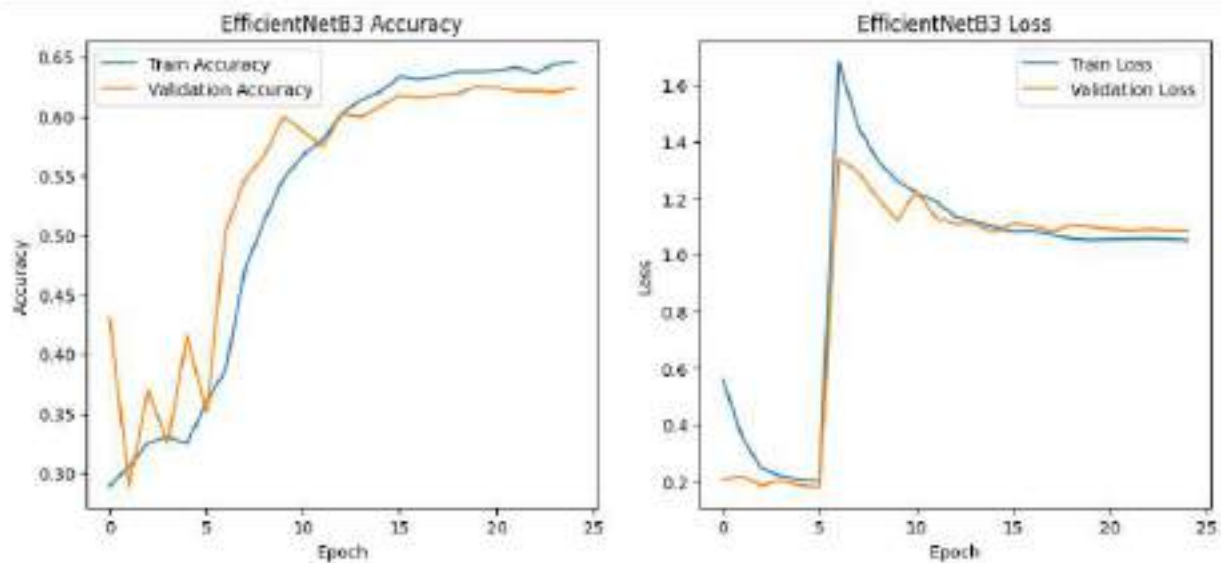


Fig.No: 7.3 Knee Osteoarthritis Detection (KL-Grade I – IV) Evaluation Plot (Loss and Accuracy plot Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using EfficientNetB3)

- **Loss Reduction**

Loss reduction is consistent and smooth throughout training. The gap between training and validation loss is minimal. This indicates strong generalization and low overfitting. The model achieves optimal performance efficiently.

ResNest50

- **Accuracy Trend**

Accuracy improves steadily but slightly lower than EfficientNetB3. Training accuracy is higher than validation accuracy. The model performs well on majority classes. Performance drops slightly for minority classes.

- **Loss Behavior**

Loss decreases consistently during training. Validation loss shows small fluctuations. This indicates moderate sensitivity to dataset imbalance. Overall convergence is stable.

- **Model Performance Insight**

Residual connections help the model learn deep features effectively. It captures general patterns in knee X-rays. However, fine-grained feature extraction is slightly limited. This affects performance on complex cases.

- **Interpretation**

The model correctly classifies most images but struggles with borderline cases. Misclassifications occur between similar KL grades. Predictions are still reliable for general classification. It performs well as a baseline model.

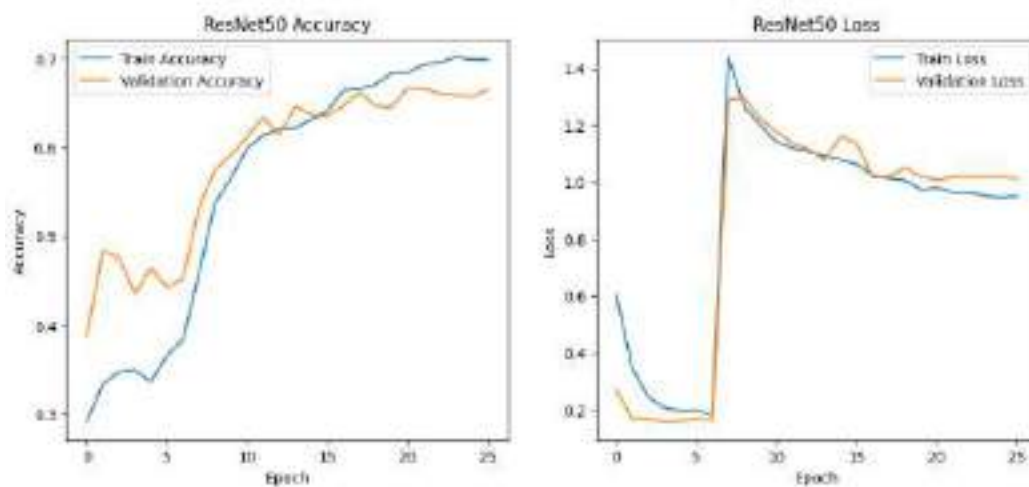


Fig.No: 7.3 Knee Osteoarthritis Detection (KL-Grade I – IV) Evaluation Plot (Loss and Accuracy plot Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using ResNet50)

- **Loss Reduction**

Loss reduces steadily but not as smoothly as EfficientNetB3. Slight gaps between training and validation loss are observed. This indicates mild overfitting. Performance remains acceptable.

DenseNet121

- **Accuracy Trends**

Accuracy improves gradually but remains lower compared to other models. Training accuracy increases steadily. Validation accuracy shows some variation. The model is sensitive to dataset imbalance.

- **Loss Behavior**

Loss decreases slowly during training. Validation loss fluctuates more compared to other models. This indicates less stable learning. Convergence is slower.

- **Model Performance Insight**

Dense connections help reuse features across layers. The model captures repetitive patterns effectively. However, it struggles with rare classes. Performance is moderate overall.

- **Interpretation**

The model correctly identifies common patterns but struggles with complex cases. Misclassification is higher for minority classes. It performs adequately but not optimally. Results are less consistent.

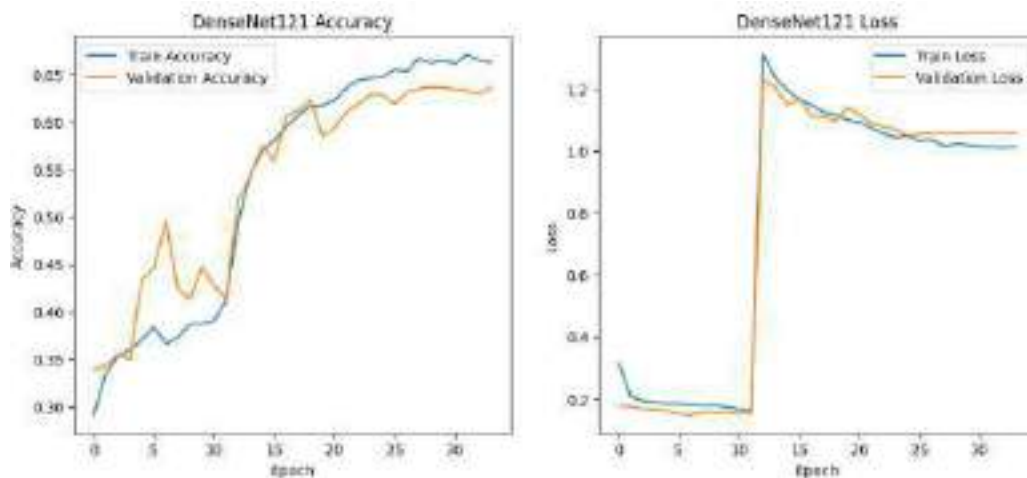


Fig.No: 7.3 Knee Osteoarthritis Detection (KL-Grade I – IV) Evaluation Plot (Loss and Accuracy plot Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using DenseNet121)

- **Loss Reduction**

Loss reduction is gradual and less stable. The gap between training and validation loss is higher. This indicates weaker generalization. Optimization is slower compared to other models.

7.4 Confusion Matrix Results

- **EfficientNetB3 Confusion Matrix Interpretation**

EfficientNetB3 shows strong classification for majority classes such as KL0 and KL2. However, it struggles with KL1 and KL4 due to limited training samples. The model tends to misclassify borderline cases between adjacent KL grades, indicating difficulty in distinguishing subtle differences.

- **ResNet50 Confusion Matrix Interpretation**

ResNet50 demonstrates better classification balance across all classes. It achieves higher correct predictions in KL0, KL2, and KL3 while reducing misclassification errors. However, KL1 still shows lower recall, indicating that early-stage osteoarthritis remains challenging to detect.

- **DenseNet121 Confusion Matrix Interpretation**

DenseNet121 performs well for dominant classes but shows slightly more confusion between KL2 and KL3. The model captures general patterns but is less effective in fine-grained classification compared to ResNet50.

CHAPTER 8

CONCLUSION

8. CONCLUSION

8.1 Conclusion

The Knee Osteoarthritis Detection system based on KL grading (I–IV) successfully demonstrates the application of deep learning in medical image classification. The project effectively utilizes three advanced CNN architectures—EfficientNetB3, ResNet50, and DenseNet121—to classify knee X-ray images into severity levels. Among these, ResNet50 achieved the best overall performance, showing higher training and validation accuracy along with better generalization on unseen data. The results confirm that deep learning models can reliably extract clinically relevant features such as joint space narrowing and bone deformities.

The experimental results also highlight important dataset-related challenges. Although all models performed well on majority classes (KL0, KL2, KL3), they showed comparatively lower performance on minority classes like KL1 due to class imbalance in the dataset. The use of class weights improved the situation, but the confusion matrix results indicate that subtle differences between adjacent KL grades remain difficult to capture. The ensemble model improved prediction stability but did not significantly outperform the best individual model, indicating that model architecture plays a more critical role than simple aggregation in this case.

Overall, the project successfully achieves its objective of building an automated, interpretable, and reasonably accurate system for knee osteoarthritis detection. The combination of model training, evaluation metrics, and visualization techniques ensures both performance and transparency, making the system suitable for real-world assistance in clinical decision-making. However, further improvements in dataset quality and model tuning are required to achieve higher diagnostic precision, especially for early-stage detection.

8.2 Key Achievements

- Successfully implemented three deep learning architectures (EfficientNetB3, ResNet50, DenseNet121) for multi-class KL grading classification.
- Achieved a maximum test accuracy of ~69.26% using ResNet50, demonstrating strong model performance on medical imaging data.
- Developed an ensemble model (~68.06% accuracy) to improve prediction robustness and reduce variance.

- Applied class weighting techniques to handle dataset imbalance, improving minority class learning.
- Generated and analyzed confusion matrices, identifying model strengths and weaknesses across KL grades.
- Integrated visual interpretation techniques (Grad-CAM) to validate that models focus on medically relevant regions.
- Built a Streamlit-based application for real-time prediction, making the system user-friendly and deployable.

8.3 Future Enhancements

- **Dataset Expansion**

The current dataset shows imbalance across KL grades, which affects model performance, especially for KL1. Future work can include collecting more labeled X-ray images, particularly for underrepresented classes, to improve model generalization and reduce bias.

- **Advanced Data Augmentation**

Although basic preprocessing is used, advanced augmentation techniques such as rotation, contrast enhancement, and noise injection can be applied. This will help the model learn more robust features and perform better on real-world variations in X-ray images.

- **Hyperparameter Optimization**

Further tuning of learning rate, batch size, optimizer selection, and fine-tuning layers can significantly improve performance. Automated techniques like Grid Search or Bayesian Optimization can be used to identify optimal configurations.

- **Use of More Advanced Architectures**

Future improvements can involve testing newer architectures such as Vision Transformers (ViTs) or hybrid CNN-transformer models. These models may capture global image dependencies better than traditional CNNs.

- **Improved Class Imbalance Handling**

Techniques like SMOTE (Synthetic Minority Oversampling), focal loss, or balanced batch sampling can be applied to improve classification performance for minority classes, especially KL1.

- **Clinical Integration and Validation**

The model can be validated with real clinical data and integrated into hospital systems. Collaboration with medical professionals can help refine predictions and improve reliability for diagnostic use.

- **Explainability and Decision Support**

Enhancing explainability using advanced visualization tools can make the system more trustworthy. Providing clear reasoning behind predictions can help doctors use the system as a decision-support tool rather than just a prediction model.

- **Multi-Modal Data Integration**

Currently, the model relies only on X-ray images for prediction. Future improvements can include integrating clinical data such as patient age, BMI, symptoms, and medical history along with imaging data. Combining these inputs using multi-modal deep learning can significantly improve diagnostic accuracy, especially for early-stage osteoarthritis where visual differences are subtle.

- **Real-Time Deployment Optimization**

To make the system more practical, the model can be optimized for real-time deployment on low-resource devices such as hospital systems or mobile platforms. Techniques like model quantization, pruning, and TensorFlow Lite conversion can reduce model size and inference time while maintaining performance.

- **Continuous Learning System (Model Updating)**

The current model is static after training. A future enhancement would be implementing a continuous learning pipeline, where the model can be periodically retrained with new patient data. This allows the system to adapt to new patterns, improve over time, and maintain high accuracy in real-world clinical environments.

CHAPTER 9

APPENDIX

9. APPENDIX

9.1 Coding

9.1.1 Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version of your notebook
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

/kaggle/input/datasets/tommyngx/kneeoa/auto_test/2/9239817_1.png
/kaggle/input/datasets/tommyngx/kneeoa/auto_test/2/9525665_1.png
/kaggle/input/datasets/tommyngx/kneeoa/auto_test/2/9483254_1.png
/kaggle/input/datasets/tommyngx/kneeoa/auto_test/2/9175691_1.png
/kaggle/input/datasets/tommyngx/kneeoa/auto_test/2/9552100_2.png
/kaggle/input/datasets/tommyngx/kneeoa/auto_test/2/9034891_1.png
/kaggle/input/datasets/tommyngx/kneeoa/auto_test/2/9634146_1.png
```

```
import tensorflow as tf
import numpy as np
import pandas as pd
import time

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import EfficientNetB3, ResNet50, DenseNet121
from tensorflow.keras.applications.efficientnet import preprocess_input as eff_pre
from tensorflow.keras.applications.resnet50 import preprocess_input as res_pre
from tensorflow.keras.applications.densenet import preprocess_input as dense_pre

from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np

train_dir = "/kaggle/input/datasets/tommyngr/kneeoa/train"

temp_gen = ImageDataGenerator().flow_from_directory(
    train_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)

print("Class indices:")
print(temp_gen.class_indices)

print("\nClass distribution:")
print(np.bincount(temp_gen.classes))

Found 5778 images belonging to 5 classes.
Class indices:
{'0': 0, '1': 1, '2': 2, '3': 3, '4': 4}

Class distribution:
[2286 1846 1516 757 173]

print("Class distribution:")
counts = np.bincount(temp_gen.classes)
print(counts)

for i, c in enumerate(counts):
    print(f"Grade {i}: {c} images")

Class distribution:
[2286 1846 1516 757 173]
Grade 0: 2286 images
Grade 1: 1846 images
Grade 2: 1516 images
Grade 3: 757 images
Grade 4: 173 images

train_dir = "/kaggle/input/datasets/tommyngr/kneeoa/train"
valid_dir = "/kaggle/input/datasets/tommyngr/kneeoa/val"
test_dir = "/kaggle/input/datasets/tommyngr/kneeoa/test"

IMG_SIZE = (300,300)
BATCH = 16
EPOCHS_STAGE1 = 30
EPOCHS_STAGE2 = 25

```

```

def focal_loss(gamma=2., alpha=0.25):
    def loss(y_true, y_pred):
        epsilon = 1e-7
        y_pred = tf.clip_by_value(y_pred, epsilon, 1. - epsilon)
        cross_entropy = -y_true * tf.math.log(y_pred)
        weight = alpha * tf.pow(1 - y_pred, gamma)
        return tf.reduce_sum(weight * cross_entropy, axis=-1)
    return loss

```



```

def train_model(model_name, base_model_class, preprocess_func):
    import tensorflow.keras.backend as K
    K.clear_session()

    print("\n===== {model_name} =====")

    size = (224,224) if model_name == "ResNet50" else IMG_SIZE

    # ----- DATA GENERATORS -----
    train_gen = ImageDataGenerator(
        preprocessing_function=preprocess_func,
        rotation_range=25,
        zoom_range=0.25,
        horizontal_flip=True,
        width_shift_range=0.1,
        height_shift_range=0.1,
        shear_range=0.1,
        fill_mode="nearest",
        brightness_range=[0.7,1.3]
    ).flow_from_directory(
        train_dir,
        target_size=size,
        batch_size=BATCH,
        class_mode="categorical"
    )

    from sklearn.utils.class_weight import compute_class_weight
    import numpy as np

    class_weights = compute_class_weight(
        class_weight="balanced",
        classes=np.unique(train_gen.classes),
        y=train_gen.classes
    )

    class_weights = dict(enumerate(class_weights))
    print("Class Weights:", class_weights)

    val_gen = ImageDataGenerator(
        preprocessing_function=preprocess_func
    ).flow_from_directory(
        valid_dir,
        target_size=size,
        batch_size=BATCH,
        class_mode="categorical"
    )

    test_gen = ImageDataGenerator(
        preprocessing_function=preprocess_func
    ).flow_from_directory(
        test_dir,
        target_size=size,
        batch_size=BATCH,
        class_mode="categorical",
        shuffle=False
    )

```

```

# ----- MODEL -----
base_model = base_model_class(
    include_top=False,
    weights="imagenet",
    input_shape=(size,3)
)

base_model.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = BatchNormalization()(x)
x = Dense(256, activation="relu")(x)
x = Dropout(0.6)(x)
output = Dense(5, activation="softmax")(x)

model = Model(base_model.input, output)

model.compile(
    optimizer="adam",
    loss=focal_loss(),
    metrics=["accuracy"]
)

callbacks = [
    EarlyStopping(patience=5, restore_best_weights=True, monitor="val_accuracy"),
    ReduceLROnPlateau(patience=2, factor=0.3)
]

start = time.time()

# ----- STAGE 1 -----
history1 = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS_STAGE1,
    callbacks=callbacks,
    class_weight=class_weights
)

# ----- STAGE 2 -----
base_model.trainable = True

for layer in base_model.layers[:120]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    #loss=focal_loss(),
    loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
    metrics=["accuracy"]
)

history2 = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS_STAGE2,

```

- Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using EfficientNetB3

```

model_eff, acc_eff, time_eff, h1_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, h1_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, h1_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)

----- EfficientNetB3 -----
Found 5778 images belonging to 5 classes.
Class Weights: {0: np.float64(0.5055118110236221), 1: np.float64(1.1047801147227534), 2: np.float64(0.7622601292875089), 3: np.float64(1.5265521796565389), 4: np.float64(6.679768786127168)}
Found 826 images belonging to 5 classes.
Found 1656 images belonging to 5 classes.

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your 'PyDataset' class should call 'super().__init__(**kwargs)' in its constructor. '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not pass these arguments to 'fit()', as they will be ignored.
  self._warn_if_super_not_called()

Epoch 1/50
362/362 ----- 187s 426ms/step - accuracy: 0.2691 - loss: 0.6282 - val_accuracy: 0.4208 - val_loss: 0.2063 - learning_rate: 0.0010
Epoch 2/50
362/362 ----- 121s 335ms/step - accuracy: 0.3001 - loss: 0.3679 - val_accuracy: 0.2001 - val_loss: 0.2474 - learning_rate: 0.0010

```

- Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using ResNet50

```

model_eff, acc_eff, time_eff, h1_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, h1_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, h1_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)

EfficientNetB3 Test Accuracy: 64.49%
Training Time: 55.93 minutes

----- ResNet50 -----
Found 5778 images belonging to 5 classes.
Class Weights: {0: np.float64(0.5055118110236221), 1: np.float64(1.1047801147227534), 2: np.float64(0.7622601292875089), 3: np.float64(1.5265521796565389), 4: np.float64(6.679768786127168)}
Found 826 images belonging to 5 classes.
Found 1656 images belonging to 5 classes.

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your 'PyDataset' class should call 'super().__init__(**kwargs)' in its constructor. '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not pass these arguments to 'fit()', as they will be ignored.
  self._warn_if_super_not_called()

Epoch 1/50
362/362 ----- 98s 241ms/step - accuracy: 0.2677 - loss: 0.6615 - val_accuracy: 0.3886 - val_loss: 0.2717 - learning_rate: 0.0010
Epoch 2/50

```

- Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using DenseNet121

```

model_eff, acc_eff, time_eff, h1_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, h1_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, h1_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)

```

ResNet50 Test Accuracy: 69.26%
Training Time: 36.58 minutes

----- DenseNet121 -----
Found 5778 images belonging to 5 classes.
Class Weights: {0: np.float64(0.5055118118236221), 1: np.float64(1.1047801147227534), 2: np.float64(0.7622691292875989), 3: np.float64(1.5265521796565389), 4: np.float64(6.679768786127168)}
Found 826 images belonging to 5 classes.
Found 1636 images belonging to 5 classes.

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your "PyDataset" class should call 'super().__init__(**kwargs)' in its constructor. '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not pass these arguments to 'fit()', as they will be ignored.
self._warn_if_super_not_called()

Epoch 1/50
362/362 — 182s 440ms/step - accuracy: 0.2668 - loss: 0.3774 - val_accuracy: 0.3300 - val_loss: 0.1785 - learning_rate: 0.0010
Epoch 2/50

- Visualizations

```
import pandas as pd
import matplotlib.pyplot as plt

# Total images per class
data = {
    "Class": ["KL0", "KL1", "KL2", "KL3", "KL4"],
    "Total Images": [3253, 1455, 2175, 1086, 251]
}

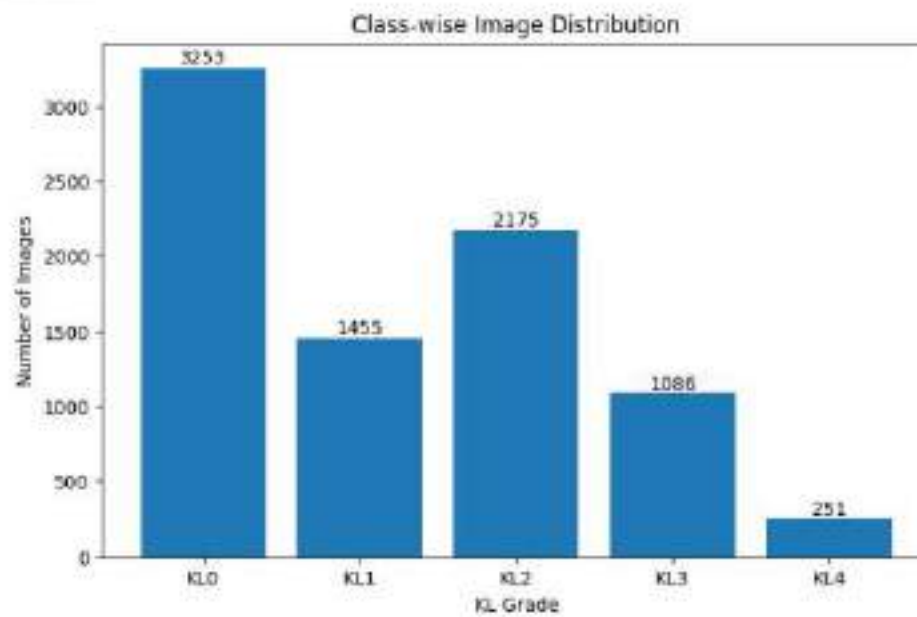
df = pd.DataFrame(data)

plt.figure(figsize=(8,9))
plt.bar(df["Class"], df["Total Images"])

plt.title("Class-wise Image Distribution")
plt.xlabel("KL Grade")
plt.ylabel("Number of Images")

# display values on bars
for i, v in enumerate(df["Total Images"]):
    plt.text(i, v + 30, str(v), ha='center')

plt.show()
```



```

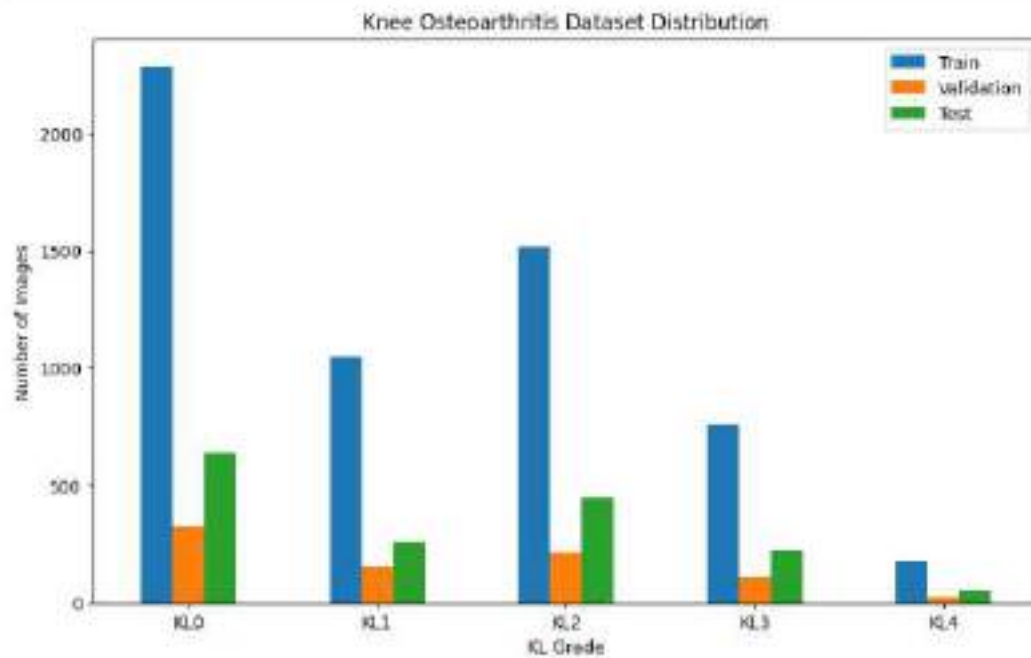
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Class": ["KL0", "KL1", "KL2", "KL3", "KL4"],
    "Train": [2286, 1046, 1516, 757, 173],
    "Validation": [328, 153, 212, 106, 37],
    "Test": [639, 256, 447, 223, 51]
}

df = pd.DataFrame(data)
df.set_index("Class").plot(kind="bar", figsize=(18,6))

plt.title("Knee Osteoarthritis Dataset Distribution")
plt.ylabel("Number of Images")
plt.xlabel("KL Grade")
plt.xticks(rotation=0)
plt.show()

```




```

# Your model accuracies
results = {
    "EfficientNetB3": acc_eff,
    "ResNet50": acc_res,
    "DenseNet121": acc_dense
}

# Extract model names and accuracies
models = list(results.keys())
accuracies = [v * 100 for v in results.values()]

# Find best model
best_model = max(results, key=results.get)

# Set colors (best model in green)
colors = ['skyblue' if m != best_model else 'green' for m in models]

# Plot bar chart
plt.figure(figsize=(8,5))
plt.bar(models, accuracies, color=colors)

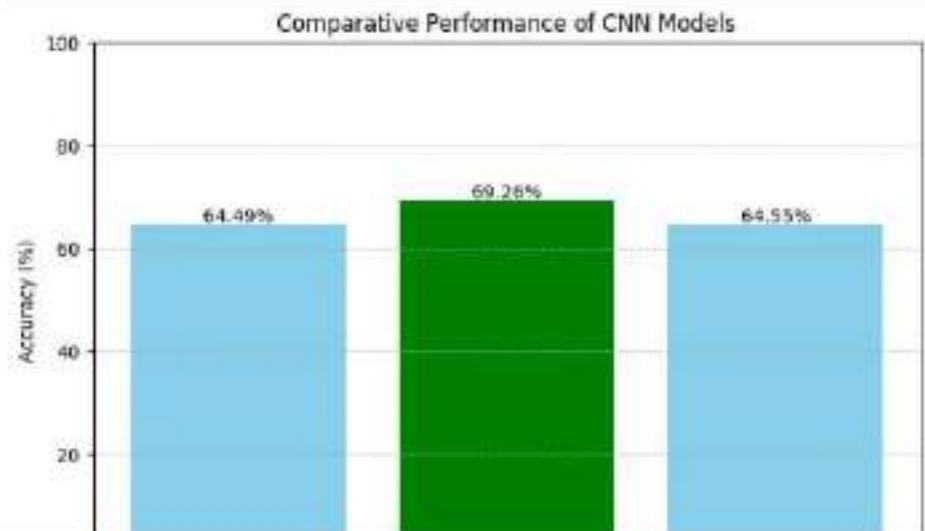
plt.title("Comparative Performance of CNN Models")
plt.xlabel("Models")
plt.ylabel("Accuracy (%)")

# Show accuracy values on bars
for i, v in enumerate(accuracies):
    plt.text(i, v + 0.5, f"{v:.2f}%", ha='center')

plt.ylim(0,100)
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()

```



9.2 Outputs

- Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using EfficientNetB3

```
model_eff, acc_eff, time_eff, hl_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, hl_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, hl_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)
```

----- EfficientNetB3 -----
Found 5778 images belonging to 5 classes.
Class Weights: {0: np.float64(0.5055118110236221), 1: np.float64(1.1047801147227534), 2: np.float64(0.7622691292875089), 3: np.float64(1.5265511796585380), 4: np.float64(6.679768786127168)}
Found 826 images belonging to 5 classes.
Found 1656 images belonging to 5 classes.

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:111: UserWarning: Your 'PyDataset' class should call 'super().__init__(**kwargs)' in its constructor. '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not pass these arguments to 'fit()', as they will be ignored.
self._warn_if_super_not_called()

Epoch 1/50
362/362 ----- 187s 426ms/step - accuracy: 0.2691 - loss: 0.6282 - val_accuracy: 0.4298 - val_loss: 0.2063 - learning_rate: 0.0010
Epoch 2/50
362/362 ----- 121s 335ms/step - accuracy: 0.3001 - loss: 0.3679 - val_accuracy: 0.2001 - val_loss: 0.1470 - learning_rate: 0.0010

model_eff, acc_eff, time_eff, hl_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, hl_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, hl_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)

247 - val_loss: 1.0909 - learning_rate: 2.7000e-05
Epoch 15/25
362/362 ----- 126s 348ms/step - accuracy: 0.6432 - loss: 1.0523 - val_accuracy: 0.6235 - val_loss: 1.0918 - learning_rate: 2.7000e-05
Epoch 16/25
362/362 ----- 124s 342ms/step - accuracy: 0.6361 - loss: 1.0539 - val_accuracy: 0.6211 - val_loss: 1.0864 - learning_rate: 8.1000e-07
Epoch 17/25
362/362 ----- 126s 347ms/step - accuracy: 0.6345 - loss: 1.0549 - val_accuracy: 0.6211 - val_loss: 1.0913 - learning_rate: 8.1000e-07
Epoch 18/25
362/362 ----- 127s 350ms/step - accuracy: 0.6388 - loss: 1.0619 - val_accuracy: 0.6199 - val_loss: 1.0869 - learning_rate: 2.4300e-07
Epoch 19/25
362/362 ----- 126s 348ms/step - accuracy: 0.6452 - loss: 1.0587 - val_accuracy: 0.6235 - val_loss: 1.0869 - learning_rate: 2.4300e-07
104/104 ----- 13s 120ms/step - accuracy: 0.7814 - loss: 0.8633
EfficientNetB3 Test Accuracy: 64.49%
Training Time: 55.93 minutes


```

import matplotlib.pyplot as plt

def plot_history(history1, history2, model_name):
    acc = history1.history['accuracy'] + history2.history['accuracy']
    val_acc = history1.history['val_accuracy'] + history2.history['val_accuracy']

    loss = history1.history['loss'] + history2.history['loss']
    val_loss = history1.history['val_loss'] + history2.history['val_loss']

    epochs = range(len(acc))

    plt.figure(figsize=(12,5))

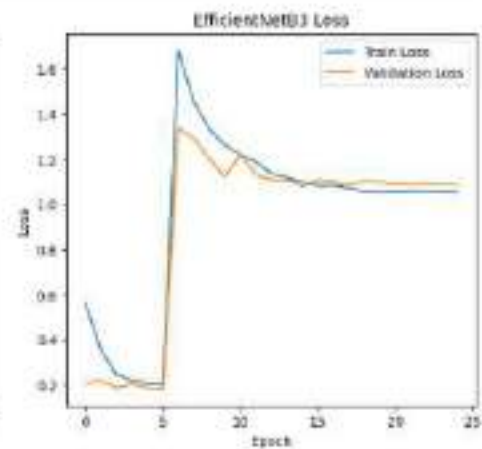
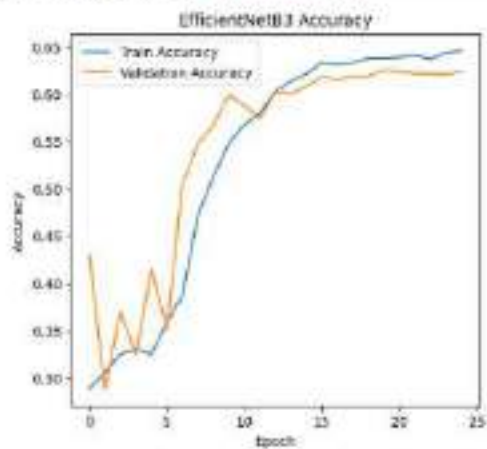
    # Accuracy
    plt.subplot(1,2,1)
    plt.plot(epochs/5, acc, label='Train Accuracy')
    plt.plot(epochs/5, val_acc, label='Validation Accuracy')
    plt.title(f'{model_name} Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    # Loss
    plt.subplot(1,2,2)
    plt.plot(epochs/5, loss, label='Train Loss')
    plt.plot(epochs/5, val_loss, label='Validation Loss')
    plt.title(f'{model_name} Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.show()

# Plot graphs for each model
plot_history(h1_acc, h2_acc, 'EfficientNetB0')
plot_history(h1_acc, h2_acc, 'ResNet50')
plot_history(h1_devise, h2_devise, 'DenseNet121')

```



```

import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

def get_predictions(model, generator):
    generator.reset()
    preds = model.predict(generator, verbose=1)
    y_pred = np.argmax(preds, axis=1)
    y_true = generator.classes
    return y_true, y_pred

def plot_conf_matrix(y_true, y_pred, model_name, class_names):

    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(6,5))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
    disp.plot(cmap="Blues", values_format='d')

    plt.title(F'{model_name} Confusion Matrix')
    plt.xticks(rotation=45)
    plt.show()

print([var for var in dir() if "gen" in var.lower()])

['ImageDataGenerator', 'temp_gen', 'test_gen_dense', 'test_gen_eff', 'test_gen_n

class_names = list(test_gen_eff.class_indices.keys())

y_true_eff, y_pred_eff = get_predictions(model_eff, test_gen_eff)
plot_conf_matrix(y_true_eff, y_pred_eff, "EfficientNetB3", class_names)

print("EfficientNetB3 Report:\n")
print(classification_report(y_true_eff, y_pred_eff, target_names=class_names))

```

EfficientNetB3 Report:

	precision	recall	F1-score	support
0	0.61	0.00	0.70	630
1	0.36	0.17	0.23	290
2	0.65	0.57	0.61	667
3	0.83	0.58	0.68	223
4	0.70	0.73	0.76	51
accuracy			0.61	1656
macro avg	0.63	0.08	0.61	1656
weighted avg	0.63	0.64	0.62	1656

- Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using ResNet50

```
model_eff, acc_eff, time_eff, h1_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, h1_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, h1_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)
```

```
----- ResNet50 -----
Found 5778 images belonging to 5 classes.
Class Weights: {0: np.float64(0.5855118110236221), 1: np.float64(1.1847801147227534), 2: np.float64(0.7622691292875989), 3: np.float64(1.5265521796565389), 4: np.float64(6.679768786127168)}
Found 826 images belonging to 5 classes.
Found 1096 images belonging to 5 classes.
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:12:
1: UserWarning: Your 'PyDataset' class should call 'super().__init__(**kwargs)' in its constructor.
  '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not pass these arguments to 'fit()', as they will be ignored.
  self._warn_if_super_not_called()
```

```
Epoch 1/50
362/362 ----- 98s 24ms/step - accuracy: 0.2677 - loss: 0.6615 - val_accuracy: 0.38
86 - val_loss: 0.2717 - learning_rate: 0.0010
Epoch 2/50
362/362 ----- 78s 214ms/step - accuracy: 0.3396 - loss: 0.3908 - val_accuracy: 0.48
43 - val_loss: 0.1091 - learning_rate: 0.0010
Epoch 3/50
```

```
model_eff, acc_eff, time_eff, h1_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, h1_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, h1_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)
```

```
71 - val_loss: 1.0860 - learning_rate: 0.0000e-06
Epoch 15/25
362/362 ----- 81s 224ms/step - accuracy: 0.7062 - loss: 0.9520 - val_accuracy: 0.66
59 - val_loss: 1.0171 - learning_rate: 0.0000e-06
Epoch 16/25
362/362 ----- 82s 225ms/step - accuracy: 0.7038 - loss: 0.9498 - val_accuracy: 0.66
18 - val_loss: 1.0202 - learning_rate: 0.0000e-06
Epoch 17/25
362/362 ----- 82s 226ms/step - accuracy: 0.7105 - loss: 0.9422 - val_accuracy: 0.65
86 - val_loss: 1.0195 - learning_rate: 2.7000e-06
Epoch 18/25
362/362 ----- 81s 223ms/step - accuracy: 0.7058 - loss: 0.9410 - val_accuracy: 0.65
74 - val_loss: 1.0281 - learning_rate: 2.7000e-06
Epoch 19/25
362/362 ----- 81s 224ms/step - accuracy: 0.7017 - loss: 0.9398 - val_accuracy: 0.66
46 - val_loss: 1.0114 - learning_rate: 8.1000e-07
104/104 ----- 9s 80ms/step - accuracy: 0.7046 - loss: 0.8532
ResNet50 Test Accuracy: 69.26%
Training Time: 36.58 minutes
```

```

import matplotlib.pyplot as plt

def plot_history(history1, history2, model_name):

    acc = history1.history['accuracy'] + history2.history['accuracy']
    val_acc = history1.history['val_accuracy'] + history2.history['val_accuracy']

    loss = history1.history['loss'] + history2.history['loss']
    val_loss = history1.history['val_loss'] + history2.history['val_loss']

    epochs = range(len(acc))

    plt.figure(figsize=(12,5))

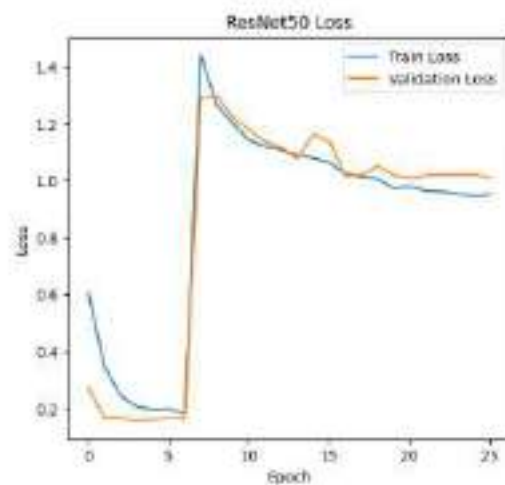
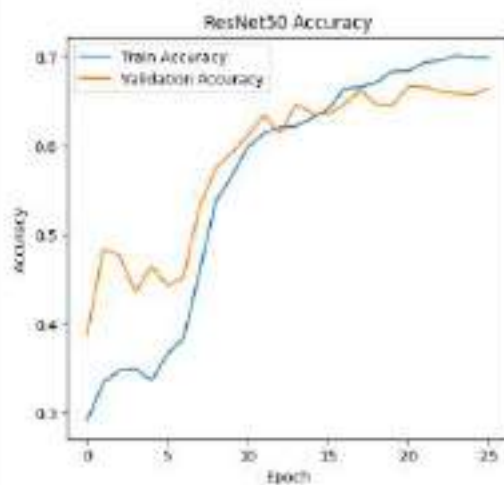
    # Accuracy
    plt.subplot(1,2,1)
    plt.plot(epochs, acc, label="Train Accuracy")
    plt.plot(epochs, val_acc, label="Validation Accuracy")
    plt.title(f"{model_name} Accuracy")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.legend()

    # Loss
    plt.subplot(1,2,2)
    plt.plot(epochs, loss, label="Train Loss")
    plt.plot(epochs, val_loss, label="Validation Loss")
    plt.title(f"{model_name} Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()

    plt.show()

# Plot graphs for each model
plot_history(h1_eff, h2_eff, "EfficientNetB3")
plot_history(h1_res, h2_res, "ResNet50")
plot_history(h1_dense, h2_dense, "DenseNet121")

```



```

class_names = list(test_gen_res.class_indices.keys())

y_true_res, y_pred_res = get_predictions(model_res, test_gen_res)
plot_conf_matrix(y_true_res, y_pred_res, "ResNet50", class_names)

print("ResNet50 Report:\n")
print(classification_report(y_true_res, y_pred_res, target_names=class_names))

```

ResNet50 Report:

	precision	recall	f1-score	support
0	0.70	0.89	0.78	639
1	0.36	0.16	0.22	296
2	0.68	0.71	0.69	447
3	0.88	0.77	0.82	223
4	0.87	0.80	0.84	51
accuracy			0.69	1656
macro avg	0.70	0.67	0.67	1656
weighted avg	0.66	0.69	0.67	1656

- Knee Osteoarthritis Dataset with KL Grading – 2018 Dataset using DenseNet121

```
model_eff, acc_eff, time_eff, h1_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, h1_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, h1_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)
```

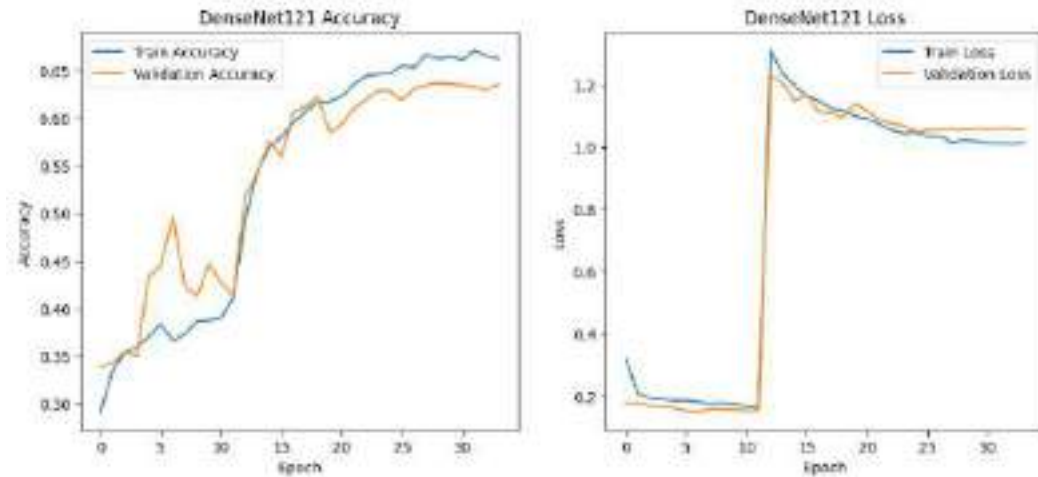
```
----- DenseNet121 -----
Found 5778 images belonging to 5 classes.
Class Weights: {0: np.float64(0.5055118110236221), 1: np.float64(1.1047801147227534), 2: np.float64(0.7612691292875909), 3: np.float64(1.5265521796565389), 4: np.float64(6.679768786127168)}
Found 826 images belonging to 5 classes.
Found 1636 images belonging to 5 classes.
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:12:
UserWarning: Your 'PyDataset' class should call 'super().__init__(**kwargs)' in its constructor. '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not pass these arguments to 'fit()', as they will be ignored.
self.warn_if_super_not_called()
```

```
Epoch 1/50
362/362 ----- 182s 440ms/step - accuracy: 0.2668 - loss: 0.3774 - val_accuracy: 0.3398 - val_loss: 0.1785 - learning_rate: 0.0010
Epoch 2/50
362/362 ----- 128s 353ms/step - accuracy: 0.3348 - loss: 0.2115 - val_accuracy: 0.3426 - val_loss: 0.1764 - learning_rate: 0.0010
Epoch 3/50
```

```
model_eff, acc_eff, time_eff, h1_eff, h2_eff = train_model("EfficientNetB3", EfficientNetB3, eff_pre)
model_res, acc_res, time_res, h1_res, h2_res = train_model("ResNet50", ResNet50, res_pre)
model_dense, acc_dense, time_dense, h1_dense, h2_dense = train_model("DenseNet121", DenseNet121, dense_pre)
```

```
368 - val_loss: 1.0579 - learning_rate: 9.0000e-06
Epoch 18/25
362/362 ----- 128s 353ms/step - accuracy: 0.6735 - loss: 1.0098 - val_accuracy: 0.6356 - val_loss: 1.0611 - learning_rate: 2.7000e-06
Epoch 19/25
362/362 ----- 129s 358ms/step - accuracy: 0.6704 - loss: 1.0005 - val_accuracy: 0.6344 - val_loss: 1.0609 - learning_rate: 2.7000e-06
Epoch 20/25
362/362 ----- 128s 352ms/step - accuracy: 0.6778 - loss: 0.9998 - val_accuracy: 0.6320 - val_loss: 1.0614 - learning_rate: 8.1000e-07
Epoch 21/25
362/362 ----- 131s 361ms/step - accuracy: 0.6705 - loss: 1.0077 - val_accuracy: 0.6295 - val_loss: 1.0611 - learning_rate: 8.1000e-07
Epoch 22/25
362/362 ----- 126s 349ms/step - accuracy: 0.6587 - loss: 1.0143 - val_accuracy: 0.6356 - val_loss: 1.0597 - learning_rate: 2.4300e-07
104/104 ----- 19s 181ms/step - accuracy: 0.7761 - loss: 0.8674
DenseNet121 Test Accuracy: 64.33%
Training Time: 75.83 minutes
```



```
class_names = list(test_gen_dense.class_indices.keys())

y_true_dense, y_pred_dense = get_predictions(model_dense, test_gen_dense)
plot_conf_matrix(y_true_dense, y_pred_dense, "DenseNet121", class_names)

print("DenseNet121 Report:\n")
print(classification_report(y_true_dense, y_pred_dense, target_names=class_names))
```

DenseNet121 Report:

	precision	recall	f1-score	support
0	0.63	0.93	0.75	639
1	0.31	0.00	0.10	296
2	0.64	0.62	0.63	447
3	0.84	0.62	0.71	223
4	0.80	0.80	0.80	51
accuracy			0.65	1656
macro avg	0.64	0.61	0.60	1656
weighted avg	0.61	0.65	0.60	1656

```

import matplotlib.pyplot as plt

# Your model accuracies
results = {
    "EfficientNetB3": acc_eff,
    "ResNet50": acc_res,
    "DenseNet121": acc_dense
}

# Extract model names and accuracies
models = list(results.keys())
accuracies = [v * 100 for v in results.values()]

# Find best model
best_model = max(results, key=results.get)

# Set colors (best model is green)
colors = ['skyblue' if m != best_model else 'green' for m in models]

# Plot bar chart
plt.figure(figsize=(8,5))
plt.bar(models, accuracies, color=colors)

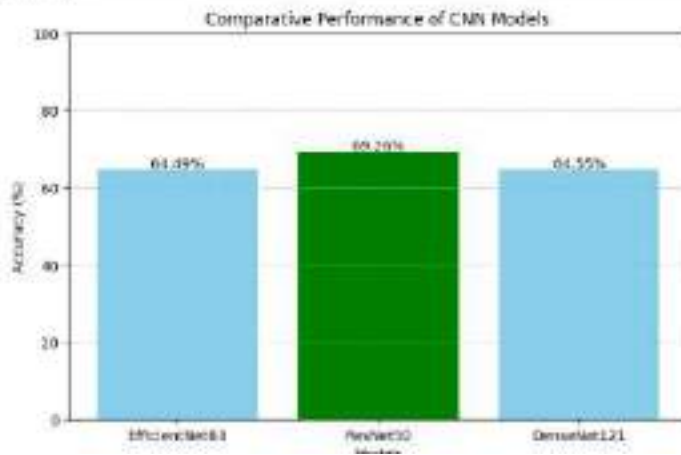
plt.title('Comparative Performance of CNN Models')
plt.xlabel('Models')
plt.ylabel('Accuracy (%)')

# Show accuracy values on bars
for i, v in enumerate(accuracies):
    plt.text(1, v + 0.5, f'{v:.2f}%', ha='center')

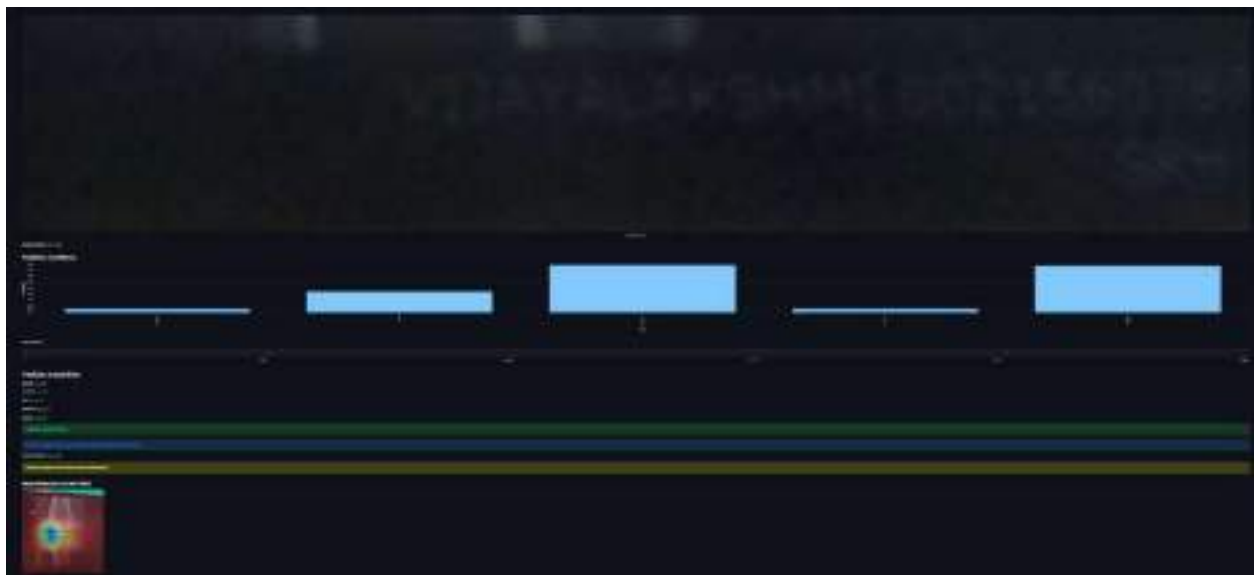
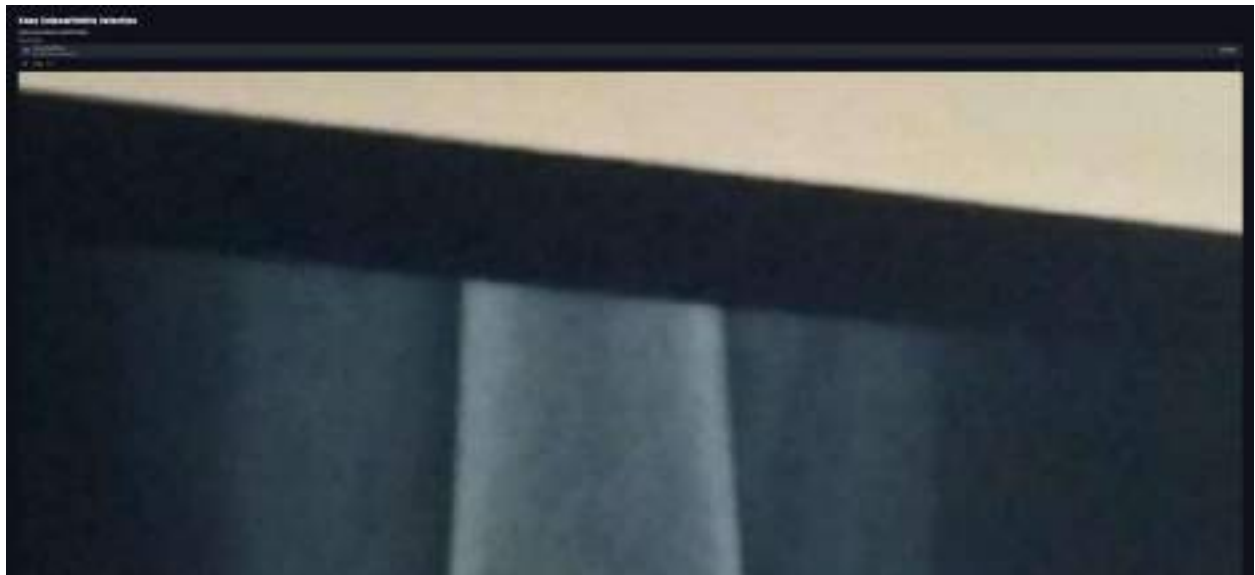
plt.ylim(0,100)
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()

```



- **Streamlit Interface**



CHAPTER 10

BIBLIOGRAPHY AND REFERENCE

10. BIBLIOGRAPHY AND REFERENCE

- <https://www.kaggle.com/code/kongkeartnara/densenet201>
- <https://www.kaggle.com/datasets/tommyngx/kneeo>
- <https://www.kaggle.com/code/aryandessai/knee-osteoarthritis>
- <https://www.kaggle.com/code/dedkoster/image-interpetation-pytorch>
- <https://www.kaggle.com/code/ghananjanisaini/osteoarthritis-kneexray>
- <https://www.kaggle.com/code/haidermasood/xception-osteoarthritis-kneexray>
- <https://www.kaggle.com/code/akashkewar/data-sprint-35-osteoarthritis-kneexray>
- <https://www.kaggle.com/code/rmehedi/kl-grading-classification-federated-learning>
- <https://www.kaggle.com/code/sherifsamyabdelkarem/knee-x-ray-cnn>